

View

App

FlagdemoApplication

+ {static} main(String[] args): void

SQLiteDialect

+ SQLiteDialect()
- registerColumnType(int varchar, String s): void
+ getIdentityColumnSupport(): IdentityColumnSupport

FlaggleView

FlaggleController

- CountryController {final}: countryController

+ @GetMapping("/") showLobby(): String
+ @GetMapping("/start") startGame(DifficultyLevel difficulty, Model model, HttpSession session): String
+ @PostMapping("/guess") guess(String countryName, String gameId, Model model, HttpSession session): String
+ @PostMapping("/giveup") giveUp(String gameId, Model model, HttpSession session): String
- buildGuessList(FlaggleViewModel viewModel): List

HomeController

+ @GetMapping("/") home(): String

GlobeView

GlobeController

- CountryController {final}: countryController

+ @GetMapping("/") showLobby(): String
+ @GetMapping("/start") startGame(Model model, HttpSession session): String
+ @RequestMapping("/guess/ajax") @ResponseBody guessAjax(String countryName, String gameId, HttpSession session): GuessResultGlobeBL
+ @PostMapping("/giveup/ajax") @ResponseBody giveUpAjax(String gameId, HttpSession session): Map
+ @PostMapping("/hint/ajax") @ResponseBody hintAjax(String gameId, HttpSession session): Map
+ @GetMapping("/countries") @ResponseBody getCountries(String gameId, HttpSession session): List
- isValidForGlobe(CountryBL country): boolean

MatchView

FlaggleBlitzController

- Set {static final}: VALID_DURATIONS_SECONDS
- CountryController {final}: countryController

+ @GetMapping("/lobby") lobby(int durationSeconds, Model model): String
+ @GetMapping("/start") start(DifficultyLevel difficulty, AiSkillLevel aiLevel, int durationSeconds, Model model, HttpSession session): String
- normalizeDuration(int durationSeconds): int
+ @PostMapping("/guess/ajax") @ResponseBody guessAjax(String matchId, String countryName, HttpSession session): Map
+ @PostMapping("/giveup/ajax") @ResponseBody giveUpAjax(String matchId, HttpSession session): Map
+ @PostMapping("/status/ajax") @ResponseBody statusAjax(String matchId, HttpSession session): Map
+ @PostMapping("/pause/ajax") @ResponseBody pauseAjax(String matchId, HttpSession session): Map
+ @PostMapping("/resume/ajax") @ResponseBody resumeAjax(String matchId, HttpSession session): Map
- buildStatePayload(FlaggleBlitzViewModel viewModel, GuessResultBL lastGuess, String givenUpName): Map
- encodeToBase64(BufferedImage image): String

FlaggleMultiplayerController

- Set {static final}: VALID_BEST_OF
- Set {static final}: VALID_DURATIONS_SECONDS
- CountryController {final}: countryController
- GameRoomRegistryBL {final}: registry

- + @GetMapping("/lobby") lobby(): String
- + @PostMapping("/create") create(DifficultyLevel difficulty, int bestOf, Model model, HttpSession session): String
- + @PostMapping("/blitz/create") createBlitz(DifficultyLevel difficulty, int durationSeconds, Model model, HttpSession session): String
- + @PostMapping("/join") join(String roomCode, Model model, HttpSession session): String
- + @GetMapping("/room") room(String roomCode, Model model, HttpSession session): String
- renderMatchScreen(PvpMatchRoom room, int slot, Model model): String
- normalizeBestOf(int bestOf): int
- normalizeDuration(int durationSeconds): int
- + @PostMapping("/guess/ajax") @ResponseBody guessAjax(String roomCode, String playerToken, String countryName): Map
- + @PostMapping("/giveup/ajax") @ResponseBody giveUpAjax(String roomCode, String playerToken): Map
- + @PostMapping("/nextRound/ajax") @ResponseBody nextRoundAjax(String roomCode, String playerToken): Map
- + @GetMapping("/status/ajax") @ResponseBody statusAjax(String roomCode, String playerToken): Map
- + @PostMapping("/pause/ajax") @ResponseBody pauseAjax(String roomCode, String playerToken): Map
- + @PostMapping("/resume/ajax") @ResponseBody resumeAjax(String roomCode, String playerToken): Map
- buildStatePayload(PvpMatchViewModel viewModel, GuessResultBL lastGuess): Map
- + @PostMapping("/blitz/guess/ajax") @ResponseBody blitzGuessAjax(String roomCode, String playerToken, String countryName): Map
- + @PostMapping("/blitz/giveup/ajax") @ResponseBody blitzGiveUpAjax(String roomCode, String playerToken): Map
- + @GetMapping("/blitz/status/ajax") @ResponseBody blitzStatusAjax(String roomCode, String playerToken): Map
- + @PostMapping("/blitz/pause/ajax") @ResponseBody blitzPauseAjax(String roomCode, String playerToken): Map
- + @PostMapping("/blitz/resume/ajax") @ResponseBody blitzResumeAjax(String roomCode, String playerToken): Map
- buildBlitzStatePayload(PvpBlitzViewModel viewModel, GuessResultBL lastGuess, String givenUpName): Map
- mapWinner(PvpRoundWinner winner, int viewerSlot): String
- encodeToBase64(BufferedImage image): String

FlaggleMatchController

- Set {static final}: VALID_BEST_OF
- CountryController {final}: countryController

- + @GetMapping("/") modeSelect(): String
- + @GetMapping("/format") format(): String
- + @GetMapping("/lobby") lobby(int bestOf, Model model): String
- + @GetMapping("/start") start(DifficultyLevel difficulty, AiSkillLevel aiLevel, int bestOf, Model model, HttpSession session): String
- normalizeBestOf(int bestOf): int
- + @PostMapping("/guess/ajax") @ResponseBody guessAjax(String matchId, String countryName, HttpSession session): Map
- + @PostMapping("/giveup/ajax") @ResponseBody giveUpAjax(String matchId, HttpSession session): Map
- + @GetMapping("/status/ajax") @ResponseBody statusAjax(String matchId, HttpSession session): Map
- + @PostMapping("/pause/ajax") @ResponseBody pauseAjax(String matchId, HttpSession session): Map
- + @PostMapping("/resume/ajax") @ResponseBody resumeAjax(String matchId, HttpSession session): Map
- + @PostMapping("/nextRound/ajax") @ResponseBody nextRoundAjax(String matchId, HttpSession session): Map
- buildStatePayload(FlaggleMatchViewModel viewModel, GuessResultBL lastGuess): Map
- encodeToBase64(BufferedImage image): String

GlobeMatchView

GlobeBlitzController

- Set {static final}: VALID_DURATIONS_SECONDS
- CountryController {final}: countryController

+ @GetMapping("/lobby") lobby(int durationSeconds, Model model): String
+ @GetMapping("/start") start(AiSkillLevel aiLevel, int durationSeconds, Model model, HttpSession session): String
- normalizeDuration(int durationSeconds): int
+ @PostMapping("/guess/ajax") @ResponseBody guessAjax(String matchId, String countryName, HttpSession session): Map
+ @PostMapping("/giveup/ajax") @ResponseBody giveUpAjax(String matchId, HttpSession session): Map
+ @GetMapping("/status/ajax") @ResponseBody statusAjax(String matchId, HttpSession session): Map
+ @PostMapping("/pause/ajax") @ResponseBody pauseAjax(String matchId, HttpSession session): Map
+ @PostMapping("/resume/ajax") @ResponseBody resumeAjax(String matchId, HttpSession session): Map
- buildStatePayload(GlobeBlitzViewModel viewModel, GuessResultGlobeBL lastGuess, String givenUpName): Map

GlobeMatchController

- Set {static final}: VALID_BEST_OF
- CountryController {final}: countryController

+ @GetMapping("/") modeSelect(): String
+ @GetMapping("/format") format(): String
+ @GetMapping("/lobby") lobby(int bestOf, Model model): String
+ @GetMapping("/start") start(AiSkillLevel aiLevel, int bestOf, Model model, HttpSession session): String
- normalizeBestOf(int bestOf): int
+ @PostMapping("/guess/ajax") @ResponseBody guessAjax(String matchId, String countryName, HttpSession session): Map
+ @PostMapping("/giveup/ajax") @ResponseBody giveUpAjax(String matchId, HttpSession session): Map
+ @GetMapping("/status/ajax") @ResponseBody statusAjax(String matchId, HttpSession session): Map
+ @PostMapping("/pause/ajax") @ResponseBody pauseAjax(String matchId, HttpSession session): Map
+ @PostMapping("/resume/ajax") @ResponseBody resumeAjax(String matchId, HttpSession session): Map
+ @PostMapping("/nextRound/ajax") @ResponseBody nextRoundAjax(String matchId, HttpSession session): Map
+ @PostMapping("/hint/ajax") @ResponseBody hintAjax(String matchId, HttpSession session): Map
- buildStatePayload(GlobeMatchViewModel viewModel, GuessResultGlobeBL lastGuess): Map

GlobeMultiplayerController

- Set {static final}: VALID_BEST_OF
- Set {static final}: VALID_DURATIONS_SECONDS
- CountryController {final}: countryController
- GlobeRoomRegistryBL {final}: registry

+ @GetMapping("/lobby") lobby(): String
+ @PostMapping("/create") create(int bestOf, Model model, HttpSession session): String
+ @PostMapping("/blitz/create") createBlitz(int durationSeconds, Model model, HttpSession session): String
+ @PostMapping("/join") join(String roomCode, Model model, HttpSession session): String
+ @GetMapping("/room") room(String roomCode, Model model, HttpSession session): String
- renderMatchScreen(GlobePvpMatchRoom room, int slot, Model model): String
- normalizeBestOf(int bestOf): int
- normalizeDuration(int durationSeconds): int
+ @PostMapping("/guess/ajax") @ResponseBody guessAjax(String roomCode, String playerToken, String countryName): Map
+ @PostMapping("/giveup/ajax") @ResponseBody giveUpAjax(String roomCode, String playerToken): Map
+ @PostMapping("/nextRound/ajax") @ResponseBody nextRoundAjax(String roomCode, String playerToken): Map
+ @PostMapping("/hint/ajax") @ResponseBody hintAjax(String roomCode, String playerToken): Map
+ @GetMapping("/status/ajax") @ResponseBody statusAjax(String roomCode, String playerToken): Map
+ @PostMapping("/pause/ajax") @ResponseBody pauseAjax(String roomCode, String playerToken): Map
+ @PostMapping("/resume/ajax") @ResponseBody resumeAjax(String roomCode, String playerToken): Map
- buildStatePayload(PvpGlobeMatchViewModel viewModel, GuessResultGlobeBL lastGuess): Map
+ @PostMapping("/blitz/guess/ajax") @ResponseBody blitzGuessAjax(String roomCode, String playerToken, String

```

+ @PostMapping("/blitz/guess/ajax") @ResponseBody blitzGuessAjax(String roomCode, String playerToken, String
countryName): Map
+ @PostMapping("/blitz/giveup/ajax") @ResponseBody blitzGiveUpAjax(String roomCode, String playerToken): Map
+ @GetMapping("/blitz/status/ajax") @ResponseBody blitzStatusAjax(String roomCode, String playerToken): Map
+ @PostMapping("/blitz/pause/ajax") @ResponseBody blitzPauseAjax(String roomCode, String playerToken): Map
+ @PostMapping("/blitz/resume/ajax") @ResponseBody blitzResumeAjax(String roomCode, String playerToken): Map
- buildBlitzStatePayload(PvpGlobeBlitzViewModel viewModel, GuessResultGlobeBL lastGuess, String givenUpName): Map
- mapWinner(PvpRoundWinner winner, int viewerSlot): String

```

CapitalView

CapitalController

```

- Set {static final}: VALID_TARGETS
- CountryController {final}: countryController

```

```

+ CapitalController(CountryController countryController)
+ @GetMapping("") hub(): String
+ @GetMapping("/{mode}/format") format(String mode, Model model): String
+ @GetMapping("/{mode}/com/format") comFormat(String mode, Model model): String
+ @GetMapping("/{mode}/lobby") lobby(String mode, int target, Model model): String
+ @GetMapping("/{mode}/start") start(String mode, AiSkillLevel aiLevel, int target, Model model, HttpSession session): String
- normalizeTarget(int target): int
- parseMode(String mode): CapitalQuizMode
- modeLabel(CapitalQuizMode mode): String
+ @PostMapping("/{mode}/answer/ajax") @ResponseBody answerAjax(String mode, String matchId, int optionIndex,
HttpSession session): Map
+ @GetMapping("/{mode}/status/ajax") @ResponseBody statusAjax(String mode, String matchId, HttpSession session): Map
+ @PostMapping("/{mode}/pause/ajax") @ResponseBody pauseAjax(String mode, String matchId, HttpSession session):
Map
+ @PostMapping("/{mode}/resume/ajax") @ResponseBody resumeAjax(String mode, String matchId, HttpSession session):
Map
+ @PostMapping("/{mode}/nextRound/ajax") @ResponseBody nextRoundAjax(String mode, String matchId, HttpSession
session): Map
- buildStatePayload(CapitalQuizViewModel viewModel): Map

```

CapitalRegularController

```

- CountryController {final}: countryController

```

```

+ CapitalRegularController(CountryController countryController)
+ @GetMapping("/{mode}/regular") regular(String mode, Model model): String
+ @GetMapping("/{mode}/regular/question/ajax") @ResponseBody questionAjax(String mode): Map
- parseMode(String mode): CapitalQuizMode
- modeLabel(CapitalQuizMode mode): String

```

CapitalBlitzController

- Set {static final}: VALID_DURATIONS_SECONDS
- CountryController {final}: countryController

+ CapitalBlitzController(CountryController countryController)
+ @GetMapping("/{mode}/blitz/lobby") lobby(String mode, int durationSeconds, Model model): String
+ @GetMapping("/{mode}/blitz/start") start(String mode, AiSkillLevel aiLevel, int durationSeconds, Model model, HttpSession session): String
- normalizeDuration(int durationSeconds): int
- parseMode(String mode): CapitalQuizMode
- modeLabel(CapitalQuizMode mode): String
+ @PostMapping("/{mode}/blitz/answer/ajax") @ResponseBody answerAjax(String mode, String matchId, int optionIndex, HttpSession session): Map
+ @GetMapping("/{mode}/blitz/status/ajax") @ResponseBody statusAjax(String mode, String matchId, HttpSession session): Map
+ @PostMapping("/{mode}/blitz/pause/ajax") @ResponseBody pauseAjax(String mode, String matchId, HttpSession session): Map
+ @PostMapping("/{mode}/blitz/resume/ajax") @ResponseBody resumeAjax(String mode, String matchId, HttpSession session): Map
- buildStatePayload(CapitalBlitzViewModel viewModel, Boolean lastAnswerCorrect): Map

CapitalMultiplayerController

- Set {static final}: VALID_BEST_OF
- Set {static final}: VALID_DURATIONS_SECONDS
- CountryController {final}: countryController
- CapitalQuizRoomRegistryBL {final}: registry

+ CapitalMultiplayerController(CountryController countryController, CapitalQuizRoomRegistryBL registry)
+ @GetMapping("/lobby") lobby(String mode, Model model): String
+ @PostMapping("/create") create(String mode, int bestOf, Model model, HttpSession session): String
+ @PostMapping("/blitz/create") createBlitz(String mode, int durationSeconds, Model model, HttpSession session): String
+ @PostMapping("/join") join(String mode, String roomCode, Model model, HttpSession session): String
+ @GetMapping("/room") room(String mode, String roomCode, Model model, HttpSession session): String
- renderMatchScreen(String mode, CapitalQuizMode quizMode, CapitalPvpMatchRoom room, int slot, Model model): String
- normalizeBestOf(int bestOf): int
- normalizeDuration(int durationSeconds): int
- parseMode(String mode): CapitalQuizMode
- modeLabel(CapitalQuizMode mode): String
+ @PostMapping("/answer/ajax") @ResponseBody answerAjax(String mode, String roomCode, String playerToken, int optionIndex): Map
+ @GetMapping("/status/ajax") @ResponseBody statusAjax(String mode, String roomCode, String playerToken): Map
+ @PostMapping("/pause/ajax") @ResponseBody pauseAjax(String mode, String roomCode, String playerToken): Map
+ @PostMapping("/resume/ajax") @ResponseBody resumeAjax(String mode, String roomCode, String playerToken): Map
- buildStatePayload(PvpCapitalQuizViewModel viewModel): Map
+ @PostMapping("/blitz/answer/ajax") @ResponseBody blitzAnswerAjax(String mode, String roomCode, String playerToken, int optionIndex): Map
+ @GetMapping("/blitz/status/ajax") @ResponseBody blitzStatusAjax(String mode, String roomCode, String playerToken): Map
+ @PostMapping("/blitz/pause/ajax") @ResponseBody blitzPauseAjax(String mode, String roomCode, String playerToken): Map
+ @PostMapping("/blitz/resume/ajax") @ResponseBody blitzResumeAjax(String mode, String roomCode, String playerToken): Map
- buildBlitzStatePayload(PvpCapitalBlitzViewModel viewModel, Boolean lastAnswerCorrect): Map
- mapWinner(PvpRoundWinner winner, int viewerSlot): String

CapitalGlobeView

CapitalGlobeController

- CountryController {final}: countryController

+ CapitalGlobeController(CountryController countryController)
+ @GetMapping("/{}/format") format(): String
+ @GetMapping("/com/format") comFormat(): String
+ @GetMapping("/start") start(Model model, HttpSession session): String
+ @RequestMapping("/guess/ajax") @ResponseBody guessAjax(String countryName, String gameId, HttpSession session): GuessResultGlobeBL
+ @PostMapping("/giveup/ajax") @ResponseBody giveUpAjax(String gameId, HttpSession session): Map

CapitalGlobeMatchController

- Set {static final}: VALID_TARGETS
- CountryController {final}: countryController

+ CapitalGlobeMatchController(CountryController countryController)
+ @GetMapping("/lobby") lobby(int target, Model model): String
+ @GetMapping("/start") start(AiSkillLevel aiLevel, int target, Model model, HttpSession session): String
- normalizeTarget(int target): int
+ @PostMapping("/guess/ajax") @ResponseBody guessAjax(String matchId, String countryName, HttpSession session): Map
+ @PostMapping("/giveup/ajax") @ResponseBody giveUpAjax(String matchId, HttpSession session): Map
+ @GetMapping("/status/ajax") @ResponseBody statusAjax(String matchId, HttpSession session): Map
+ @PostMapping("/pause/ajax") @ResponseBody pauseAjax(String matchId, HttpSession session): Map
+ @PostMapping("/resume/ajax") @ResponseBody resumeAjax(String matchId, HttpSession session): Map
+ @PostMapping("/nextRound/ajax") @ResponseBody nextRoundAjax(String matchId, HttpSession session): Map
- buildStatePayload(CapitalGlobeMatchViewModel viewModel, GuessResultGlobeBL lastGuess): Map

CapitalGlobeBlitzController

- Set {static final}: VALID_DURATIONS_SECONDS
- CountryController {final}: countryController

+ CapitalGlobeBlitzController(CountryController countryController)
+ @GetMapping("/lobby") lobby(int durationSeconds, Model model): String
+ @GetMapping("/start") start(AiSkillLevel aiLevel, int durationSeconds, Model model, HttpSession session): String
- normalizeDuration(int durationSeconds): int
+ @PostMapping("/guess/ajax") @ResponseBody guessAjax(String matchId, String countryName, HttpSession session): Map
+ @PostMapping("/skip/ajax") @ResponseBody skipAjax(String matchId, HttpSession session): Map
+ @GetMapping("/status/ajax") @ResponseBody statusAjax(String matchId, HttpSession session): Map
+ @PostMapping("/pause/ajax") @ResponseBody pauseAjax(String matchId, HttpSession session): Map
+ @PostMapping("/resume/ajax") @ResponseBody resumeAjax(String matchId, HttpSession session): Map
- buildStatePayload(CapitalGlobeBlitzViewModel viewModel, GuessResultGlobeBL lastGuess, String skippedName): Map

CapitalGlobeMultiplayerController

- Set {static final}: VALID_BEST_OF
- Set {static final}: VALID_DURATIONS_SECONDS
- CountryController {final}: countryController
- CapitalGlobeRoomRegistryBL {final}: registry

- + CapitalGlobeMultiplayerController(CountryController countryController, CapitalGlobeRoomRegistryBL registry)
- + @GetMapping("/lobby") lobby(): String
- + @PostMapping("/create") create(int bestOf, Model model, HttpSession session): String
- + @PostMapping("/blitz/create") createBlitz(int durationSeconds, Model model, HttpSession session): String
- + @PostMapping("/join") join(String roomCode, Model model, HttpSession session): String
- + @GetMapping("/room") room(String roomCode, Model model, HttpSession session): String
- renderMatchScreen(CapitalGlobePvpMatchRoom room, int slot, Model model): String
- normalizeBestOf(int bestOf): int
- normalizeDuration(int durationSeconds): int
- + @PostMapping("/guess/ajax") @ResponseBody guessAjax(String roomCode, String playerToken, String countryName): Map
- + @PostMapping("/giveup/ajax") @ResponseBody giveUpAjax(String roomCode, String playerToken): Map
- + @PostMapping("/nextRound/ajax") @ResponseBody nextRoundAjax(String roomCode, String playerToken): Map
- + @GetMapping("/status/ajax") @ResponseBody statusAjax(String roomCode, String playerToken): Map
- + @PostMapping("/pause/ajax") @ResponseBody pauseAjax(String roomCode, String playerToken): Map
- + @PostMapping("/resume/ajax") @ResponseBody resumeAjax(String roomCode, String playerToken): Map
- buildStatePayload(PvpCapitalGlobeMatchViewModel viewModel, GuessResultGlobeBL lastGuess): Map
- + @PostMapping("/blitz/guess/ajax") @ResponseBody blitzGuessAjax(String roomCode, String playerToken, String countryName): Map
- + @PostMapping("/blitz/skip/ajax") @ResponseBody blitzSkipAjax(String roomCode, String playerToken): Map
- + @GetMapping("/blitz/status/ajax") @ResponseBody blitzStatusAjax(String roomCode, String playerToken): Map
- + @PostMapping("/blitz/pause/ajax") @ResponseBody blitzPauseAjax(String roomCode, String playerToken): Map
- + @PostMapping("/blitz/resume/ajax") @ResponseBody blitzResumeAjax(String roomCode, String playerToken): Map
- buildBlitzStatePayload(PvpCapitalGlobeBlitzViewModel viewModel, GuessResultGlobeBL lastGuess, String skippedName): Map
- mapWinner(PvpRoundWinner winner, int viewerSlot): String

DailyQuizView

DailyQuizController

- CountryController {final}: countryController

- + DailyQuizController(CountryController countryController)
- + @GetMapping("/start") start(Integer devCountryId, String devDate, Model model, HttpServletRequest request, HttpSession session): String
- + @PostMapping("/guess") @ResponseBody guess(String countryName, String gameId, HttpSession session): Map
- + @PostMapping("/giveup") @ResponseBody giveUp(String gameId, HttpSession session): Map
- beginStage2(CountryBL target, String gameId, HttpSession session): Map
- + @PostMapping("/stage2/guess") @ResponseBody stage2Guess(String countryName, String gameId, HttpSession session, HttpServletResponse response): Map
- + @PostMapping("/stage2/giveup") @ResponseBody stage2GiveUp(String gameId, HttpSession session, HttpServletResponse response): Map
- countryPayload(CountryBL c): Map
- beginStage3(CountryBL target, String gameId, HttpSession session): Map
- + @GetMapping("/debug/today") @ResponseBody debugToday(Integer devCountryId, String devDate, HttpServletRequest request, HttpServletResponse response): Map
- + @PostMapping("/debug/reset") @ResponseBody debugReset(HttpServletResponse response): Map

ViewModel

FlaggleVM

FlaggleViewModel

- FlaggleModel: fm

+ StartNewGame(): void
+ StartNewGame(DifficultyLevel difficulty): void
+ getDifficulty(): DifficultyLevel
+ Guess(String guessesCountryName): BufferedImage
+ getAttempts(): int
+ getAllCountries(): List
+ getGuesses(): List
+ getTargetCountry(): CountryBL
+ setTargetCountry(CountryBL country): void
+ isCorrect(): boolean

GlobeVM

GlobeViewModel

- GlobeModel: gm

+ StartNewGame(): void
+ Guess(String guessedCountryName):
GuessResultGlobeBL
+ useHint(): String
+ GetAttempts(): int
+ getHintsUsed(): int
+ getAllCountries(): List
+ getGuesses(): List
+ getTargetCountry(): CountryBL
+ isCorrect(): boolean

MatchVM

FlaggleBlitzViewModel

- FlaggleBlitzModel {final}: fm

+ startMatch(): void
+ humanGuess(String countryName): GuessResultBL
+ humanGiveUpFlag(): String
+ refreshAiProgress(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ getEngine(): FlaggleBlitzEngineBL
+ getAllCountries(): List

FlaggleMatchViewModel

- FlaggleMatchModel {final}: fm

+ startMatch(): void
+ advanceToNextRound(): void
+ humanGuess(String countryName): GuessResultBL
+ humanGiveUpRound(): void
+ refreshAiTimeout(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ getEngine(): FlaggleMatchEngineBL
+ getAllCountries(): List

PvpBlitzViewModel

- PvpMatchRoom {final}: room
- int {final}: mySlot

+ getEngine(): PvpBlitzEngineBL
+ submitGuess(String countryName): GuessResultBL
+ giveUpFlag(): String

PvpMatchViewModel

- PvpMatchRoom {final}: room
- int {final}: mySlot

+ getEngine(): PvpMatchEngineBL
+ submitGuess(String countryName): GuessResultBL
+ giveUpRound(): void
+ advanceToNextRound(): void
+ markReady(): void

GlobeMatchVM

GlobeBlitzViewModel

- GlobeBlitzModel {final}: gm

+ startMatch(): void
+ humanGuess(String countryName): GuessResultGlobeBL
+ humanGiveUpFlag(): String
+ refreshAiProgress(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ getEngine(): GlobeBlitzEngineBL
+ getAllCountries(): List

GlobeMatchViewModel

- GlobeMatchModel {final}: gm

+ startMatch(): void
+ advanceToNextRound(): void
+ humanGuess(String countryName): GuessResultGlobeBL
+ humanGiveUpRound(): void
+ refreshAiTimeout(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ useHint(): String
+ getEngine(): GlobeMatchEngineBL
+ getAllCountries(): List

PvpGlobeBlitzViewModel

- GlobePvpMatchRoom {final}: room
- int {final}: mySlot

+ getEngine(): PvpGlobeBlitzEngineBL
+ submitGuess(String countryName): GuessResultGlobeBL
+ giveUpFlag(): String

PvpGlobeMatchViewModel

- GlobePvpMatchRoom {final}: room
- int {final}: mySlot

+ getEngine(): PvpGlobeMatchEngineBL
+ submitGuess(String countryName): GuessResultGlobeBL
+ giveUpRound(): void
+ advanceToNextRound(): void
+ markReady(): void
+ useHint(): String

CapitalVM

CapitalQuizViewModel

- CapitalQuizModel {final}: cm

+ CapitalQuizViewModel(CountryController cc, CapitalQuizMode mode, AiSkillLevel aiLevel, int pointsToWin)
+ startMatch(): void
+ advanceToNextRound(): void
+ submitAnswer(int optionIndex): boolean
+ pauseMatch(): void
+ resumeMatch(): void
+ getEngine(): CapitalQuizEngineBL

CapitalBlitzViewModel

- CapitalBlitzModel {final}: cm

+ CapitalBlitzViewModel(CountryController cc, CapitalQuizMode mode, AiSkillLevel aiLevel, double durationSeconds)
+ startMatch(): void
+ submitAnswer(int optionIndex): Boolean
+ refreshAiProgress(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ getEngine(): CapitalBlitzEngineBL

PvpCapitalQuizViewModel

- CapitalPvpMatchRoom {final}: room
- int {final}: mySlot

+ PvpCapitalQuizViewModel(CapitalPvpMatchRoom room, int mySlot)
+ getRoom(): CapitalPvpMatchRoom
+ getMySlot(): int
+ getEngine(): PvpCapitalQuizEngineBL
+ submitAnswer(int optionIndex): void

PvpCapitalBlitzViewModel

- CapitalPvpMatchRoom {final}: room
- int {final}: mySlot

+ PvpCapitalBlitzViewModel(CapitalPvpMatchRoom room, int mySlot)
+ getRoom(): CapitalPvpMatchRoom
+ getMySlot(): int
+ getEngine(): PvpCapitalBlitzEngineBL
+ submitAnswer(int optionIndex): Boolean

CapitalGlobeVM

CapitalGlobeViewModel

- CapitalGlobeModel {final}: cm

+ CapitalGlobeViewModel(CountryController cc)
+ startNewGame(): void
+ guess(String countryName): GuessResultGlobeBL
+ giveUp(): void
+ getEngine(): CapitalGlobeEngineBL

CapitalGlobeMatchViewModel

- CapitalGlobeMatchModel {final}: cm

+ CapitalGlobeMatchViewModel(CountryController cc, AiSkillLevel aiLevel, int pointsToWin)
+ startMatch(): void
+ advanceToNextRound(): void
+ humanGuess(String countryName): GuessResultGlobeBL
+ giveUpRound(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ getEngine(): CapitalGlobeMatchEngineBL

CapitalGlobeBlitzViewModel

- CapitalGlobeBlitzModel {final}: cm

+ CapitalGlobeBlitzViewModel(CountryController cc, AiSkillLevel aiLevel, double durationSeconds)
+ startMatch(): void
+ humanGuess(String countryName): GuessResultGlobeBL
+ humanSkipTarget(): String
+ refreshAiProgress(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ getEngine(): CapitalGlobeBlitzEngineBL

PvpCapitalGlobeMatchViewModel

- CapitalGlobePvpMatchRoom {final}: room
- int {final}: mySlot

+ PvpCapitalGlobeMatchViewModel(CapitalGlobePvpMatchRoom room, int mySlot)
+ getRoom(): CapitalGlobePvpMatchRoom
+ getMySlot(): int
+ getEngine(): PvpCapitalGlobeMatchEngineBL
+ submitGuess(String countryName): GuessResultGlobeBL
+ giveUpRound(): void
+ advanceToNextRound(): void
+ markReady(): void

PvpCapitalGlobeBlitzViewModel

- CapitalGlobePvpMatchRoom {final}: room
- int {final}: mySlot

+ PvpCapitalGlobeBlitzViewModel(CapitalGlobePvpMatchRoom room, int mySlot)
+ getRoom(): CapitalGlobePvpMatchRoom
+ getMySlot(): int
+ getEngine(): PvpCapitalGlobeBlitzEngineBL
+ submitGuess(String countryName): GuessResultGlobeBL
+ skipTarget(): String

Model

FlaggleModel

FlaggleModel

- GameService: gs
- CountryBL: targetCountry
- List: allCountries
- List: guesses
- int: attempts

+ StartNewGame(): void
+ StartNewGame(DifficultyLevel difficulty): void
+ getDifficulty(): DifficultyLevel
+ Guess(String guessesCountryName): GuessResultBL
+ resetGuesses(): void
+ setTargetCountry(CountryBL country): void

GlobeModel

GlobeModel

- GlobeGameService: gs
- CountryBL: targetCountry
- List: allCountries
- List: guesses
- int: attempts
- int: hintsUsed

+ StartNewGame(): void
+ Guess(String guessedCountryName):
GuessResultGlobeBL
+ useHint(): String
+ resetGuesses(): void

MatchModel

FlaggleBlitzModel

- FlaggleBlitzService {final}: service
- List {final}: allCountries

+ startMatch(): void
+ humanGuess(String countryName): GuessResultBL
+ humanGiveUpFlag(): String
+ refreshAiProgress(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ getEngine(): FlaggleBlitzEngineBL

FlaggleMatchModel

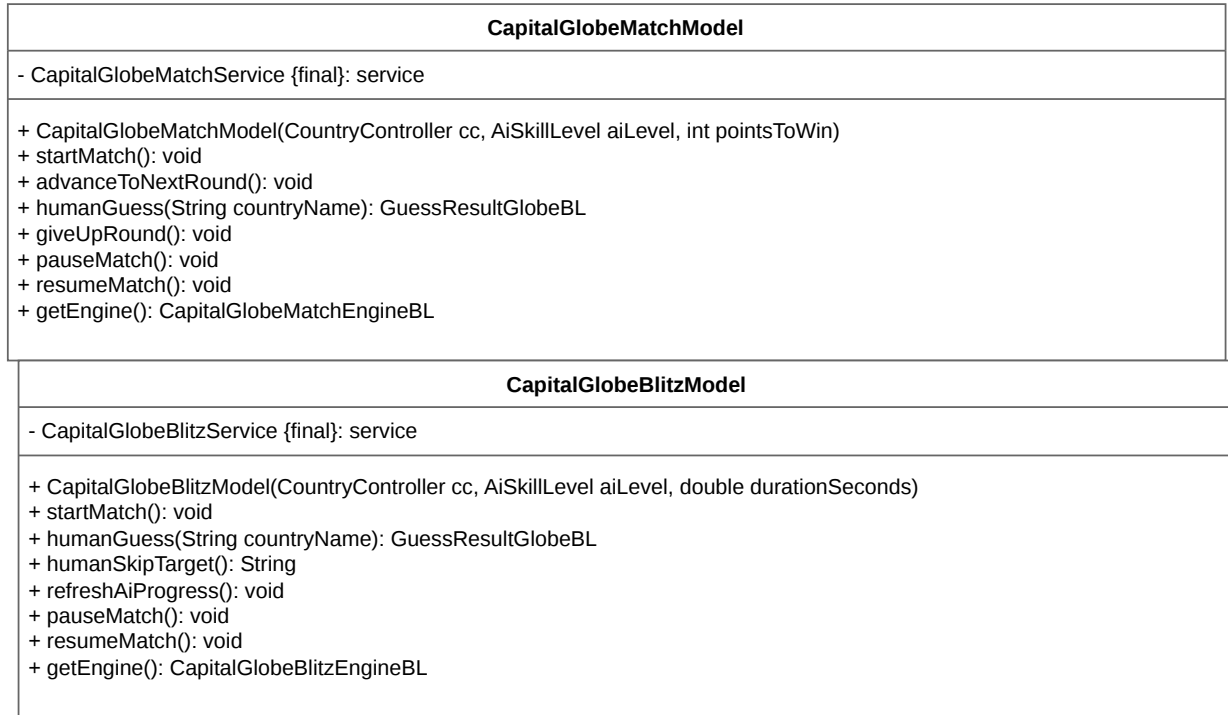
- FlaggleMatchService {final}: service
- List {final}: allCountries

+ startMatch(): void
+ advanceToNextRound(): void
+ humanGuess(String countryName): GuessResultBL
+ humanGiveUpRound(): void
+ refreshAiTimeout(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ getEngine(): FlaggleMatchEngineBL

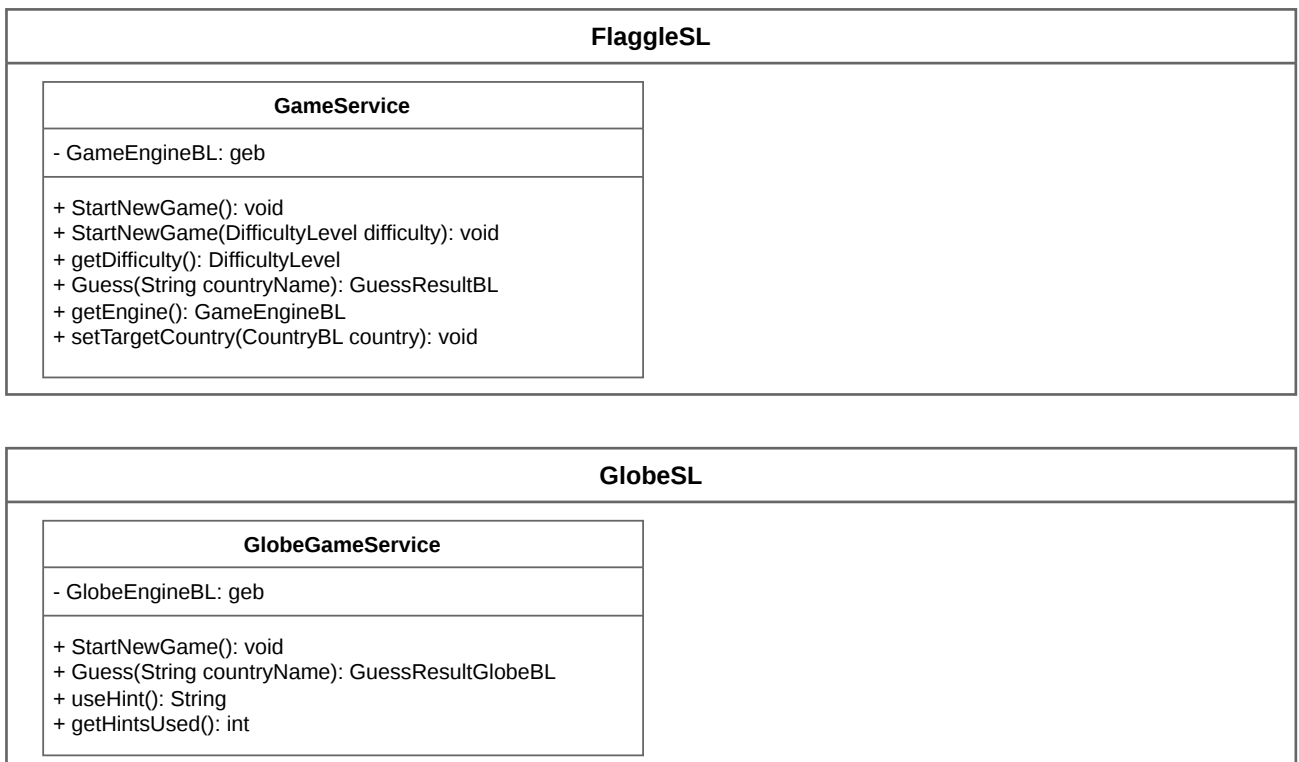
GlobeMatchModel							
<table><tr><th>GlobeBlitzModel</th></tr><tr><td> - GlobeBlitzService {final}: service - List {final}: allCountries </td></tr><tr><td> + startMatch(): void + humanGuess(String countryName): GuessResultGlobeBL + humanGiveUpFlag(): String + refreshAiProgress(): void + pauseMatch(): void + resumeMatch(): void + getEngine(): GlobeBlitzEngineBL </td></tr></table>	GlobeBlitzModel	- GlobeBlitzService {final}: service - List {final}: allCountries	+ startMatch(): void + humanGuess(String countryName): GuessResultGlobeBL + humanGiveUpFlag(): String + refreshAiProgress(): void + pauseMatch(): void + resumeMatch(): void + getEngine(): GlobeBlitzEngineBL	<table><tr><th>GlobeMatchModel</th></tr><tr><td> - GlobeMatchService {final}: service - List {final}: allCountries </td></tr><tr><td> + startMatch(): void + advanceToNextRound(): void + humanGuess(String countryName): GuessResultGlobeBL + humanGiveUpRound(): void + refreshAiTimeout(): void + pauseMatch(): void + resumeMatch(): void + useHint(): String + getEngine(): GlobeMatchEngineBL </td></tr></table>	GlobeMatchModel	- GlobeMatchService {final}: service - List {final}: allCountries	+ startMatch(): void + advanceToNextRound(): void + humanGuess(String countryName): GuessResultGlobeBL + humanGiveUpRound(): void + refreshAiTimeout(): void + pauseMatch(): void + resumeMatch(): void + useHint(): String + getEngine(): GlobeMatchEngineBL
GlobeBlitzModel							
- GlobeBlitzService {final}: service - List {final}: allCountries							
+ startMatch(): void + humanGuess(String countryName): GuessResultGlobeBL + humanGiveUpFlag(): String + refreshAiProgress(): void + pauseMatch(): void + resumeMatch(): void + getEngine(): GlobeBlitzEngineBL							
GlobeMatchModel							
- GlobeMatchService {final}: service - List {final}: allCountries							
+ startMatch(): void + advanceToNextRound(): void + humanGuess(String countryName): GuessResultGlobeBL + humanGiveUpRound(): void + refreshAiTimeout(): void + pauseMatch(): void + resumeMatch(): void + useHint(): String + getEngine(): GlobeMatchEngineBL							

CapitalModel			
<table> <tr> <th>CapitalQuizModel</th></tr> <tr> <td> - CapitalQuizService {final}: service </td></tr> <tr> <td> + CapitalQuizModel(CountryController cc, CapitalQuizMode mode, AiSkillLevel aiLevel, int pointsToWin) + startMatch(): void + advanceToNextRound(): void + submitAnswer(int optionIndex): boolean + pauseMatch(): void + resumeMatch(): void + getEngine(): CapitalQuizEngineBL </td></tr> </table>	CapitalQuizModel	- CapitalQuizService {final}: service	+ CapitalQuizModel(CountryController cc, CapitalQuizMode mode, AiSkillLevel aiLevel, int pointsToWin) + startMatch(): void + advanceToNextRound(): void + submitAnswer(int optionIndex): boolean + pauseMatch(): void + resumeMatch(): void + getEngine(): CapitalQuizEngineBL
CapitalQuizModel			
- CapitalQuizService {final}: service			
+ CapitalQuizModel(CountryController cc, CapitalQuizMode mode, AiSkillLevel aiLevel, int pointsToWin) + startMatch(): void + advanceToNextRound(): void + submitAnswer(int optionIndex): boolean + pauseMatch(): void + resumeMatch(): void + getEngine(): CapitalQuizEngineBL			
<table> <tr> <th>CapitalBlitzModel</th></tr> <tr> <td> - CapitalBlitzService {final}: service </td></tr> <tr> <td> + CapitalBlitzModel(CountryController cc, CapitalQuizMode mode, AiSkillLevel aiLevel, double durationSeconds) + startMatch(): void + submitAnswer(int optionIndex): Boolean + refreshAiProgress(): void + pauseMatch(): void + resumeMatch(): void + getEngine(): CapitalBlitzEngineBL </td></tr> </table>	CapitalBlitzModel	- CapitalBlitzService {final}: service	+ CapitalBlitzModel(CountryController cc, CapitalQuizMode mode, AiSkillLevel aiLevel, double durationSeconds) + startMatch(): void + submitAnswer(int optionIndex): Boolean + refreshAiProgress(): void + pauseMatch(): void + resumeMatch(): void + getEngine(): CapitalBlitzEngineBL
CapitalBlitzModel			
- CapitalBlitzService {final}: service			
+ CapitalBlitzModel(CountryController cc, CapitalQuizMode mode, AiSkillLevel aiLevel, double durationSeconds) + startMatch(): void + submitAnswer(int optionIndex): Boolean + refreshAiProgress(): void + pauseMatch(): void + resumeMatch(): void + getEngine(): CapitalBlitzEngineBL			

CapitalGlobeModel			
<table> <tr> <th>CapitalGlobeModel</th></tr> <tr> <td> - CapitalGlobeService {final}: service </td></tr> <tr> <td> + CapitalGlobeModel(CountryController cc) + startNewGame(): void + guess(String countryName): GuessResultGlobeBL + giveUp(): void + getEngine(): CapitalGlobeEngineBL </td></tr> </table>	CapitalGlobeModel	- CapitalGlobeService {final}: service	+ CapitalGlobeModel(CountryController cc) + startNewGame(): void + guess(String countryName): GuessResultGlobeBL + giveUp(): void + getEngine(): CapitalGlobeEngineBL
CapitalGlobeModel			
- CapitalGlobeService {final}: service			
+ CapitalGlobeModel(CountryController cc) + startNewGame(): void + guess(String countryName): GuessResultGlobeBL + giveUp(): void + getEngine(): CapitalGlobeEngineBL			



Service Layer



MatchSL							
<table><tr><th>FlaggleBlitzService</th></tr><tr><td>- FlaggleBlitzEngineBL {final}: engine</td></tr><tr><td>+ startMatch(): void + humanGuess(String countryName): GuessResultBL + humanGiveUpFlag(): String + refreshAiProgress(): void + pauseMatch(): void + resumeMatch(): void</td></tr></table>	FlaggleBlitzService	- FlaggleBlitzEngineBL {final}: engine	+ startMatch(): void + humanGuess(String countryName): GuessResultBL + humanGiveUpFlag(): String + refreshAiProgress(): void + pauseMatch(): void + resumeMatch(): void	<table><tr><th>FlaggleMatchService</th></tr><tr><td>- FlaggleMatchEngineBL {final}: engine</td></tr><tr><td>+ startMatch(): void + advanceToNextRound(): void + humanGuess(String countryName): GuessResultBL + humanGiveUpRound(): void + refreshAiTimeout(): void + pauseMatch(): void + resumeMatch(): void</td></tr></table>	FlaggleMatchService	- FlaggleMatchEngineBL {final}: engine	+ startMatch(): void + advanceToNextRound(): void + humanGuess(String countryName): GuessResultBL + humanGiveUpRound(): void + refreshAiTimeout(): void + pauseMatch(): void + resumeMatch(): void
FlaggleBlitzService							
- FlaggleBlitzEngineBL {final}: engine							
+ startMatch(): void + humanGuess(String countryName): GuessResultBL + humanGiveUpFlag(): String + refreshAiProgress(): void + pauseMatch(): void + resumeMatch(): void							
FlaggleMatchService							
- FlaggleMatchEngineBL {final}: engine							
+ startMatch(): void + advanceToNextRound(): void + humanGuess(String countryName): GuessResultBL + humanGiveUpRound(): void + refreshAiTimeout(): void + pauseMatch(): void + resumeMatch(): void							

GlobeMatchSL							
<table><tr><th>GlobeBlitzService</th></tr><tr><td>- GlobeBlitzEngineBL {final}: engine</td></tr><tr><td>+ startMatch(): void + humanGuess(String countryName): GuessResultGlobeBL + humanGiveUpFlag(): String + refreshAiProgress(): void + pauseMatch(): void + resumeMatch(): void</td></tr></table>	GlobeBlitzService	- GlobeBlitzEngineBL {final}: engine	+ startMatch(): void + humanGuess(String countryName): GuessResultGlobeBL + humanGiveUpFlag(): String + refreshAiProgress(): void + pauseMatch(): void + resumeMatch(): void	<table><tr><th>GlobeMatchService</th></tr><tr><td>- GlobeMatchEngineBL {final}: engine</td></tr><tr><td>+ startMatch(): void + advanceToNextRound(): void + humanGuess(String countryName): GuessResultGlobeBL + humanGiveUpRound(): void + refreshAiTimeout(): void + pauseMatch(): void + resumeMatch(): void + useHint(): String</td></tr></table>	GlobeMatchService	- GlobeMatchEngineBL {final}: engine	+ startMatch(): void + advanceToNextRound(): void + humanGuess(String countryName): GuessResultGlobeBL + humanGiveUpRound(): void + refreshAiTimeout(): void + pauseMatch(): void + resumeMatch(): void + useHint(): String
GlobeBlitzService							
- GlobeBlitzEngineBL {final}: engine							
+ startMatch(): void + humanGuess(String countryName): GuessResultGlobeBL + humanGiveUpFlag(): String + refreshAiProgress(): void + pauseMatch(): void + resumeMatch(): void							
GlobeMatchService							
- GlobeMatchEngineBL {final}: engine							
+ startMatch(): void + advanceToNextRound(): void + humanGuess(String countryName): GuessResultGlobeBL + humanGiveUpRound(): void + refreshAiTimeout(): void + pauseMatch(): void + resumeMatch(): void + useHint(): String							

CapitalSL			
<table><tr><th>CapitalQuizService</th></tr><tr><td>- CapitalQuizEngineBL {final}: engine</td></tr><tr><td>+ CapitalQuizService(CountryController cc, CapitalQuizMode mode, AiSkillLevel aiLevel, int pointsToWin) + startMatch(): void + advanceToNextRound(): void + submitAnswer(int optionIndex): boolean + pauseMatch(): void + resumeMatch(): void + getEngine(): CapitalQuizEngineBL</td></tr></table>	CapitalQuizService	- CapitalQuizEngineBL {final}: engine	+ CapitalQuizService(CountryController cc, CapitalQuizMode mode, AiSkillLevel aiLevel, int pointsToWin) + startMatch(): void + advanceToNextRound(): void + submitAnswer(int optionIndex): boolean + pauseMatch(): void + resumeMatch(): void + getEngine(): CapitalQuizEngineBL
CapitalQuizService			
- CapitalQuizEngineBL {final}: engine			
+ CapitalQuizService(CountryController cc, CapitalQuizMode mode, AiSkillLevel aiLevel, int pointsToWin) + startMatch(): void + advanceToNextRound(): void + submitAnswer(int optionIndex): boolean + pauseMatch(): void + resumeMatch(): void + getEngine(): CapitalQuizEngineBL			
<table><tr><th>CapitalBlitzService</th></tr><tr><td>- CapitalBlitzEngineBL {final}: engine</td></tr><tr><td>+ CapitalBlitzService(CountryController cc, CapitalQuizMode mode, AiSkillLevel aiLevel, double durationSeconds) + startMatch(): void + submitAnswer(int optionIndex): Boolean + refreshAiProgress(): void + pauseMatch(): void + resumeMatch(): void + getEngine(): CapitalBlitzEngineBL</td></tr></table>	CapitalBlitzService	- CapitalBlitzEngineBL {final}: engine	+ CapitalBlitzService(CountryController cc, CapitalQuizMode mode, AiSkillLevel aiLevel, double durationSeconds) + startMatch(): void + submitAnswer(int optionIndex): Boolean + refreshAiProgress(): void + pauseMatch(): void + resumeMatch(): void + getEngine(): CapitalBlitzEngineBL
CapitalBlitzService			
- CapitalBlitzEngineBL {final}: engine			
+ CapitalBlitzService(CountryController cc, CapitalQuizMode mode, AiSkillLevel aiLevel, double durationSeconds) + startMatch(): void + submitAnswer(int optionIndex): Boolean + refreshAiProgress(): void + pauseMatch(): void + resumeMatch(): void + getEngine(): CapitalBlitzEngineBL			

CapitalGlobeSL

CapitalGlobeService

- CapitalGlobeEngineBL {final}: engine

+ CapitalGlobeService(CountryController cc)
+ startNewGame(): void
+ guess(String countryName): GuessResultGlobeBL
+ giveUp(): void
+ getEngine(): CapitalGlobeEngineBL

CapitalGlobeMatchService

- CapitalGlobeMatchEngineBL {final}: engine

+ CapitalGlobeMatchService(CountryController cc, AiSkillLevel aiLevel, int pointsToWin)
+ startMatch(): void
+ advanceToNextRound(): void
+ humanGuess(String countryName): GuessResultGlobeBL
+ giveUpRound(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ getEngine(): CapitalGlobeMatchEngineBL

CapitalGlobeBlitzService

- CapitalGlobeBlitzEngineBL {final}: engine

+ CapitalGlobeBlitzService(CountryController cc, AiSkillLevel aiLevel, double durationSeconds)
+ startMatch(): void
+ humanGuess(String countryName): GuessResultGlobeBL
+ humanSkipTarget(): String
+ refreshAiProgress(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ getEngine(): CapitalGlobeBlitzEngineBL

Business Layer

CountryBL

- String: name
- int: ID
- String: flagPath
- double: latitude
- double: longitude
- String: neighbors
- String: iso3
- String: capital

+ CountryBL(String name, int ID, String flagPath, double lat, double longi, String neighbors, String iso3, String capital)
+ CountryBL()
+ CountryBL(CountryDAL cd)
+ getName(): String
+ getCode(): int
+ getFlagPath(): String
+ @JsonIgnore getFlagImage(): BufferedImage
+ getLatitude(): double
+ getLongitude(): double
+ getNeighborsList(): String
+ getIso3(): String
+ getCapital(): String
+ hasCapital(): boolean
+ equals(Object o): boolean (OVERRIDE)
+ hashCode(): int (OVERRIDE)

FlaggleBL

DifficultyLevel (enum)

<> DifficultyLevel
EASY
HARD

GameEngineBL

- CountryBL: targetCountry
- int: attempts
- boolean: gameOver
- CountryController: cc
- DifficultyLevel: difficulty
- BufferedImage: revealedFlag

+ GameEngineBL(CountryController cc)
+ StartNewGame(): void
+ StartNewGame(DifficultyLevel difficulty): void
+ getDifficulty(): DifficultyLevel
+ Guess(String countryName): GuessResultBL
+ IsGameOver(): boolean
+ GetAttempts(): int
+ getCountryController(): CountryController
- selectRandomCountry(): CountryBL
+ getTargetCountry(): CountryBL
+ setTargetCountry(CountryBL cb): void
+ getCc(): CountryController

GuessResultBL
- int {static final}: TOLERANCE - Color {static final}: FRAME_COLOR - Color {static final}: BACKGROUND_COLOR - Color {static final}: RED - Color {static final}: BLUE - Color {static final}: YELLOW - Color {static final}: GREEN - Color {static final}: ORANGE - Color {static final}: PURPLE - Color {static final}: BLACK - Color {static final}: WHITE - Color {static final}: GRAY - boolean {final}: isCorrect - CountryBL {final}: guessedCountry - CountryBL {final}: targetCountry - BufferedImage {final}: flagDifferences - String {final}: guessedFlagBase64 - String {final}: flagDifferencesBase64
+ toString(): String - {static} encodeToBase64(BufferedImage image): String + {static} calculateFlagDifferences(BufferedImage guessed, BufferedImage target): BufferedImage - {static} calculateAccumulatedEasyReveal(BufferedImage previousReveal, BufferedImage guessed, BufferedImage target): BufferedImage + {static} calculateMatchPercentage(BufferedImage diffImage, boolean easyMode): int - {static} normalizeColor(Color c): Color - {static} colorDistance(Color c1, Color c2): double - {static} areColorsSimilar(Color c1, Color c2): boolean - {static} isFrameOrBackground(Color c): boolean

DailyQuizBL							
<table><tr><th>DailyQuizDevConfig</th></tr><tr><td>+ boolean {static final}: DEV_MODE_ENABLED</td></tr><tr><td>- DailyQuizDevConfig()</td></tr></table>	DailyQuizDevConfig	+ boolean {static final}: DEV_MODE_ENABLED	- DailyQuizDevConfig()	<table><tr><th>DailyQuizPlayedGuardBL</th></tr><tr><td>- String {static final}: COOKIE_NAME</td></tr><tr><td>- DailyQuizPlayedGuardBL() + {static} hasPlayedToday(HttpServletRequest request): boolean + {static} markPlayedToday(HttpServletResponse response): void + {static} clearPlayed(HttpServletResponse response): void</td></tr></table>	DailyQuizPlayedGuardBL	- String {static final}: COOKIE_NAME	- DailyQuizPlayedGuardBL() + {static} hasPlayedToday(HttpServletRequest request): boolean + {static} markPlayedToday(HttpServletResponse response): void + {static} clearPlayed(HttpServletResponse response): void
DailyQuizDevConfig							
+ boolean {static final}: DEV_MODE_ENABLED							
- DailyQuizDevConfig()							
DailyQuizPlayedGuardBL							
- String {static final}: COOKIE_NAME							
- DailyQuizPlayedGuardBL() + {static} hasPlayedToday(HttpServletRequest request): boolean + {static} markPlayedToday(HttpServletResponse response): void + {static} clearPlayed(HttpServletResponse response): void							
<table><tr><th>DailyQuizSelection</th></tr><tr><td>- CountryBL {final}: country - String {final}: eventTitle - Integer {final}: eventYear - String {final}: infoText - LocalDate {final}: date - boolean {final}: devOverride</td></tr><tr><td>+ DailyQuizSelection(CountryBL country, String eventTitle, Integer eventYear, String infoText, LocalDate date, boolean devOverride) + getCountry(): CountryBL + getEventTitle(): String + getEventYear(): Integer + getInfoText(): String + getDate(): LocalDate + isDevOverride(): boolean</td></tr></table>		DailyQuizSelection	- CountryBL {final}: country - String {final}: eventTitle - Integer {final}: eventYear - String {final}: infoText - LocalDate {final}: date - boolean {final}: devOverride	+ DailyQuizSelection(CountryBL country, String eventTitle, Integer eventYear, String infoText, LocalDate date, boolean devOverride) + getCountry(): CountryBL + getEventTitle(): String + getEventYear(): Integer + getInfoText(): String + getDate(): LocalDate + isDevOverride(): boolean			
DailyQuizSelection							
- CountryBL {final}: country - String {final}: eventTitle - Integer {final}: eventYear - String {final}: infoText - LocalDate {final}: date - boolean {final}: devOverride							
+ DailyQuizSelection(CountryBL country, String eventTitle, Integer eventYear, String infoText, LocalDate date, boolean devOverride) + getCountry(): CountryBL + getEventTitle(): String + getEventYear(): Integer + getInfoText(): String + getDate(): LocalDate + isDevOverride(): boolean							

DailyQuizSelectorBL

- String {static final}: EVENTS_URL
- Duration {static final}: FETCH_TIMEOUT
- ObjectMapper {static final}: MAPPER
- LocalDate {static volatile}: cachedFetchDate
- JsonNode {static volatile}: cachedDays

- DailyQuizSelectorBL()
- + {static} getTodaysSelection(CountryController cc, Integer devOverrideCountryId): DailyQuizSelection
- + {static} getTodaysSelection(CountryController cc, Integer devOverrideCountryId, LocalDate devOverrideDate): DailyQuizSelection
- {static} tryAgentSelection(CountryController cc, LocalDate today): DailyQuizSelection
- {static synchronized} getDaysMap(LocalDate today): JsonNode
- {static} pickDeterministic(CountryController cc, LocalDate date): CountryBL

DailyQuizNeighborGuessResult

- CountryBL {final}: guessedCountry
- boolean {final}: isNeighbor
- boolean {final}: alreadyFound
- int {final}: remainingCount
- int {final}: totalCount
- boolean {final}: stageComplete

- + DailyQuizNeighborGuessResult(CountryBL guessedCountry, boolean isNeighbor, boolean alreadyFound, int remainingCount, int totalCount, boolean stageComplete)
- + getGuessedCountry(): CountryBL
- + isNeighbor(): boolean
- + isAlreadyFound(): boolean
- + getRemainingCount(): int
- + getTotalCount(): int
- + isStageComplete(): boolean

DailyQuizStage2EngineBL

- double {static final}: EARTH_RADIUS_KM
- int {static final}: NEAREST_FALLBACK_COUNT
- CountryBL {final}: targetCountry
- List {final}: neighborsToFind
- boolean {final}: usedNearestFallback
- Set {final}: found

- + DailyQuizStage2EngineBL(CountryController cc, CountryBL targetCountry)
- {static} resolveRealNeighbors(CountryController cc, String neighborIso3s): List
- {static} nearestCountries(CountryController cc, CountryBL target, int count): List
- {static} distanceKm(CountryBL a, CountryBL b): double
- + guess(String countryName, CountryController cc): DailyQuizNeighborGuessResult
- + giveUp(): List
- + isComplete(): boolean
- + getTargetCountry(): CountryBL
- + getNeighborsToFind(): List
- + getFound(): Set
- + isUsedNearestFallback(): boolean

GlobeBL

GlobeEngineBL

- CountryBL: targetCountry
- int: attempts
- boolean: gameOver
- CountryController: cc
- int: hintsUsed
- Set {final}: revealedLetterPositions

+ StartNewGame(): void
+ Guess(String countryName): GuessResultGlobeBL
+ useHint(): String
+ SelectRandomCountry(): CountryBL
- isValidForGlobe(CountryBL country): boolean
+ getAllCountries(): List

ProximityLevel (enum)

<> ProximityLevel
CORRECT
NEIGHBOR
VERY_CLOSE
CLOSE
MEDIUM
FAR
VERY_FAR
EXTREME
- String {final}: colorHex

GuessResultGlobeBL

- boolean {final}: isCorrect
- CountryBL {final}: guessedCountry
- CountryBL {final}: targetCountry
- float {final}: distance
- ProximityLevel {final}: proximityLevel
- double {static final}: EARTH_RADIUS_KM

+ getColorHex(): String
- calculateProximityLevel(): ProximityLevel
- isNeighbor(): boolean
- calculateDistance(double lat1, double lon1, double lat2, double lon2): float
+ toString(): String

CountryNameHintUtil

- Random {static final}: RANDOM

+ {static} countRevealableLetters(String name): int
+ {static} revealRandomLetter(String name, Set revealedPositions): void
+ {static} maskName(String name, Set revealedPositions): String

MatchBL

AiOpponentPlan

- List {final}: attemptTimestamps
- List {final}: displayedPercents

- + getSolveTimeSeconds(): double
- + getAttemptsSoFar(double elapsedSeconds): int
- + isSolved(double elapsedSeconds): boolean
- + getProgressPercent(double elapsedSeconds): int

AiOpponentPlanFactory

- Random {static final}: RANDOM

- + {static} createPlan(AiSkillLevel level, CountryController cc, CountryBL target, DifficultyLevel flagRevealDifficulty): AiOpponentPlan
- {static} pickCandidates(List pool, CountryBL target, int count, int sampleSize): List
- {static} computeDisplayedPercents(List candidates, CountryBL target, DifficultyLevel flagRevealDifficulty): List
- {static} hardMatchPercent(CountryBL guessed, CountryBL target): int

ScoredCandidate

- CountryBL {final}: country
- int {final}: percent

AiSkillLevel (enum)

<> AiSkillLevel
EASY
MEDIUM
HARD

BlitzFlagResult

- int {final}: order
- String {final}: countryName
- boolean {final}: solved
- int {final}: attempts
- double {final}: timeTakenSeconds

FlaggleBlitzEngineBL

- CountryController {final}: cc
- DifficultyLevel {final}: difficulty
- AiSkillLevel {final}: aiLevel
- double {final}: durationSeconds
- List: queue
- long: matchStartTimeMillis
- boolean: matchOver
- boolean: paused
- long: pauseStartedAtMillis
- long: pausedMillisTotal
- int: humanQueueIndex
- int: humanAttemptsThisFlag
- double: humanCurrentFlagStartElapsed
- BufferedImage: humanRevealedFlag
- int: humanCorrectCount
- List {final}: humanHistory
- int: aiQueueIndex
- double: aiCurrentFlagStartElapsed
- AiOpponentPlan: aiPlan
- int: aiCorrectCount
- List {final}: aiHistory

MatchRoundResult

- int {final}: roundNumber
- RoundWinner {final}: winner
- String {final}: targetCountryName
- int {final}: humanAttempts
- int {final}: aiAttempts

- + {synchronized} startMatch(): void
- + {synchronized} humanGuess(String countryName): GuessResultBL
- + {synchronized} humanGiveUpFlag(): String
- advanceHumanToNextFlag(): void
- + {synchronized} refreshAiProgress(): void
- + {synchronized} pauseMatch(): void
- + {synchronized} resumeMatch(): void
- + getElapsedSeconds(): double
- + getTimeRemainingSeconds(): double
- + getCurrentHumanTarget(): CountryBL
- + getQueueFlagAt(int index): CountryBL
- + getAiElapsedOnCurrentFlag(): double
- + getMatchWinner(): RoundWinner

FlaggleMatchEngineBL
- CountryController {final}: cc - DifficultyLevel {final}: difficulty - AiSkillLevel {final}: aiLevel - int {final}: pointsToWin - int: humanScore - int: aiScore - int: roundNumber - boolean: matchOver - long: matchStartTimeMillis - long: roundStartTimeMillis - boolean: paused - long: pauseStartedAtMillis - long: pausedMillisThisRound - long: pausedMillisTotalMatch - CountryBL: currentTarget - int: humanAttemptsThisRound - BufferedImage: humanRevealedFlag - RoundWinner: currentRoundWinner - AiOpponentPlan: aiPlan - List {final}: roundHistory
+ {synchronized} startMatch(): void + {synchronized} advanceToNextRound(): void - startNextRound(): void - selectRandomCountry(): CountryBL + {synchronized} humanGuess(String countryName): GuessResultBL + {synchronized} humanGiveUpRound(): void + {synchronized} refreshAiTimeout(): void - finishRound(): void + {synchronized} pauseMatch(): void + {synchronized} resumeMatch(): void + getRoundElapsedSeconds(): double + getMatchElapsedSeconds(): double + getBestOf(): int + isRoundOver(): boolean + getMatchWinner(): RoundWinner

PvpGameMode (enum)
<> PvpGameMode - BEST_OF_N - BLITZ

GameRoomRegistryBL
- long {static final}: STALE_ROOM_MILLIS - String {static final}: CODE_CHARS - int {static final}: CODE_LENGTH - CountryController {final}: cc - Map {final}: rooms - Random {final}: random
+ createRoom(DifficultyLevel difficulty, int bestOf, String player1Token): PvpMatchRoom + createBlitzRoom(DifficultyLevel difficulty, int durationSeconds, String player1Token): PvpMatchRoom - generateCode(): String + getRoom(String roomCode): PvpMatchRoom + removeRoom(String roomCode): void + sweepStaleRooms(): void

PvpMatchEngineBL

- CountryController {final}: cc
- DifficultyLevel {final}: difficulty
- int {final}: pointsToWin
- int: score1
- int: score2
- int: roundNumber
- boolean: matchOver
- long: matchStartTimeMillis
- long: roundStartTimeMillis
- long {static final}: PAUSE_MAX_MILLIS
- boolean: paused
- int: pausedBySlot
- long: pauseStartedAtMillis
- long: pausedMillisThisRound
- long: pausedMillisTotalMatch
- boolean: pause1Used
- boolean: pause2Used
- long {static final}: READY_MAX_MILLIS
- boolean: ready1
- boolean: ready2
- long: roundOverAtMillis
- CountryBL: currentTarget
- int: attempts1
- int: attempts2
- BufferedImage: revealedFlag1
- BufferedImage: revealedFlag2
- int: progressPercent1
- int: progressPercent2
- PvpRoundWinner: currentRoundWinner
- List {final}: roundHistory

- + {synchronized} startMatch(): void
- + {synchronized} advanceToNextRound(): void
- startNextRound(): void
- selectRandomCountry(): CountryBL
- + {synchronized} submitGuess(int slot, String countryName): GuessResultBL
- + {synchronized} giveUpRound(int slot): void
- finishRound(): void
- + {synchronized} markReady(int slot): boolean
- + isReady(int slot): boolean
- readyDeadlinePassed(): boolean
- + getReadyRemainingMillis(): long
- + {synchronized} pauseMatch(int slot): boolean
- + {synchronized} resumeMatch(): void
- + {synchronized} refreshPauseTimeout(): void
- + hasUsedPause(int slot): boolean
- + getPauseRemainingMillis(): long
- + getRoundElapsedSeconds(): double
- + getMatchElapsedSeconds(): double
- + getScore(int slot): int
- + getAttempts(int slot): int
- + getProgressPercent(int slot): int
- + getBestOf(): int
- + isRoundOver(): boolean
- + getMatchWinner(): PvpRoundWinner

PvpMatchRoom
- String {final}: roomCode - DifficultyLevel {final}: difficulty - PvpGameMode {final}: gameMode - int {final}: bestOf - int {final}: pointsToWin - int {final}: durationSeconds - CountryController {final}: cc - String {final}: player1Token - String {volatile}: player2Token - PvpRoomStatus {volatile}: status - long {volatile}: lastActivityMillis - PvpMatchEngineBL {volatile}: engine - PvpBlitzEngineBL {volatile}: blitzEngine
+ {static} createBestOf(String roomCode, DifficultyLevel difficulty, int bestOf, String player1Token, CountryController cc): PvpMatchRoom + {static} createBlitz(String roomCode, DifficultyLevel difficulty, int durationSeconds, String player1Token, CountryController cc): PvpMatchRoom + {synchronized} joinPlayer2(String token): boolean + slotForToken(String token): int + touch(): void + isStale(long staleMillis): boolean + getStatus(): PvpRoomStatus

PvpRoundResult	RoundWinner (enum)
- int {final}: roundNumber - PvpRoundWinner {final}: winner - String {final}: targetCountryName - int {final}: player1Attempts - int {final}: player2Attempts	<> RoundWinner NONE HUMAN AI DRAW

PvpBlitzEngineBL
- CountryController {final}: cc - DifficultyLevel {final}: difficulty - double {final}: durationSeconds - List: queue - long: matchStartTimeMillis - boolean: matchOver - long {static final}: PAUSE_MAX_MILLIS - boolean: paused - int: pausedBySlot - long: pauseStartedAtMillis - long: pausedMillisTotal - boolean: pause1Used - boolean: pause2Used - int: queueIndex1 - int: attemptsThisFlag1 - double: currentFlagStartElapsed1 - BufferedImage: revealedFlag1 - int: progressPercent1 - int: correctCount1 - List {final}: history1 - int: queueIndex2 - int: attemptsThisFlag2 - double: currentFlagStartElapsed2 - BufferedImage: revealedFlag2 - int: progressPercent2 - int: correctCount2 - List {final}: history2
+ {synchronized} startMatch(): void + {synchronized} submitGuess(int slot, String countryName): GuessResultBL + {synchronized} giveUpFlag(int slot): String - advanceToNextFlag(int slot): void + {synchronized} refreshState(): void + {synchronized} pauseMatch(int slot): boolean + {synchronized} resumeMatch(): void + {synchronized} refreshPauseTimeout(): void + hasUsedPause(int slot): boolean + getPauseRemainingMillis(): long + getElapsedSeconds(): double + getTimeRemainingSeconds(): double + getCorrectCount(int slot): int + getAttemptsThisFlag(int slot): int + getProgressPercent(int slot): int + getHistory(int slot): List + getCurrentTarget(int slot): CountryBL + getQueueFlagAt(int index): CountryBL + getMatchWinner(): PvpRoundWinner

PvpRoomStatus (enum)
<> PvpRoomStatus WAITING_FOR_OPPONENT IN_PROGRESS FINISHED

PvpRoundWinner (enum)
<> PvpRoundWinner NONE PLAYER1 PLAYER2 DRAW

GlobeMatchBL

GlobeAiOpponentPlan

- List {final}: attemptTimestamps
- List {final}: displayedProximity

+ getSolveTimeSeconds(): double
+ getAttemptsSoFar(double elapsedSeconds): int
+ isSolved(double elapsedSeconds): boolean
+ getProgressLevel(double elapsedSeconds): ProximityLevel

GlobeAiOpponentPlanFactory

- Random {static final}: RANDOM

+ {static} createPlan(AiSkillLevel level, CountryController cc, CountryBL target): GlobeAiOpponentPlan
- {static} sampleAttempts(int minAttempts, int maxAttempts): int
- {static} validPool(CountryController cc, CountryBL target): List
- {static} pickCandidates(List pool, CountryBL target, int count, int sampleSize): List
- {static} isCloser(GuessResultGlobeBL a, GuessResultGlobeBL b): boolean
- {static} computeDisplayedProximity(List candidates, CountryBL target): List

GlobeBlitzEngineBL

- CountryController {final}: cc
- AiSkillLevel {final}: aiLevel
- double {final}: durationSeconds
- List: queue
- long: matchStartTimeMillis
- boolean: matchOver
- boolean: paused
- long: pauseStartedAtMillis
- long: pausedMillisTotal
- int: humanQueueIndex
- int: humanAttemptsThisFlag
- double: humanCurrentFlagStartElapsed
- int: humanCorrectCount
- List {final}: humanHistory
- int: aiQueueIndex
- double: aiCurrentFlagStartElapsed
- GlobeAiOpponentPlan: aiPlan
- int: aiCorrectCount
- List {final}: aiHistory

+ {synchronized} startMatch(): void
+ {synchronized} humanGuess(String countryName): GuessResultGlobeBL
+ {synchronized} humanGiveUpFlag(): String
- advanceHumanToNextFlag(): void
+ {synchronized} refreshAiProgress(): void
+ {synchronized} pauseMatch(): void
+ {synchronized} resumeMatch(): void
+ getElapsedSeconds(): double
+ getTimeRemainingSeconds(): double
+ getCurrentHumanTarget(): CountryBL
+ getQueueFlagAt(int index): CountryBL
+ getAiElapsedOnCurrentFlag(): double
+ getAiProgressLevel(): ProximityLevel
+ getMatchWinner(): RoundWinner

GlobeMatchEngineBL

- CountryController {final}: cc
- AiSkillLevel {final}: aiLevel
- int {final}: pointsToWin
- int: humanScore
- int: aiScore
- int: roundNumber
- boolean: matchOver
- long: matchStartTimeMillis
- long: roundStartTimeMillis
- boolean: paused
- long: pauseStartedAtMillis
- long: pausedMillisThisRound
- long: pausedMillisTotalMatch
- int {static final}: GRACE_SECONDS_PER_HINT
- int: hintsUsedThisRound
- Set {final}: revealedLetterPositions
- CountryBL: currentTarget
- int: humanAttemptsThisRound
- RoundWinner: currentRoundWinner
- GlobeAiOpponentPlan: aiPlan
- RoundWinner: provisionalWinner
- double: graceDeadlineElapsedSeconds
- double: lastRoundDurationSeconds
- List {final}: roundHistory

- + {synchronized} startMatch(): void
- + {synchronized} advanceToNextRound(): void
- startNextRound(): void
- selectRandomCountry(): CountryBL
- + {synchronized} humanGuess(String countryName): GuessResultGlobeBL
- + {synchronized} humanGiveUpRound(): void
- + {synchronized} useHint(): String
- + getHintMaskedName(): String
- + getGraceRemainingSeconds(): double
- + {synchronized} refreshAiTimeout(): void
- finishRound(): void
- + {synchronized} pauseMatch(): void
- + {synchronized} resumeMatch(): void
- + getRoundElapsedSeconds(): double
- + getMatchElapsedSeconds(): double
- + getBestOf(): int
- + isRoundOver(): boolean
- + getMatchWinner(): RoundWinner
- + getAiProgressLevel(): ProximityLevel

GlobePvpMatchRoom

- String {final}: roomCode
- PvpGameMode {final}: gameMode
- int {final}: bestOf
- int {final}: pointsToWin
- int {final}: durationSeconds
- CountryController {final}: cc
- String {final}: player1Token
- String {volatile}: player2Token
- PvpRoomStatus {volatile}: status
- long {volatile}: lastActivityMillis
- PvpGlobeMatchEngineBL {volatile}: engine
- PvpGlobeBlitzEngineBL {volatile}: blitzEngine

- + {static} createBestOf(String roomCode, int bestOf, String player1Token, CountryController cc): GlobePvpMatchRoom
- + {static} createBlitz(String roomCode, int durationSeconds, String player1Token, CountryController cc): GlobePvpMatchRoom
- + {synchronized} joinPlayer2(String token): boolean
- + slotForToken(String token): int
- + touch(): void
- + isStale(long staleMillis): boolean
- + getStatus(): PvpRoomStatus

GlobeRoomRegistryBL
- long {static final}: STALE_ROOM_MILLIS - String {static final}: CODE_CHARS - int {static final}: CODE_LENGTH - CountryController {final}: cc - Map {final}: rooms - Random {final}: random
+ createRoom(int bestOf, String player1Token): GlobePvpMatchRoom + createBlitzRoom(int durationSeconds, String player1Token): GlobePvpMatchRoom - generateCode(): String + getRoom(String roomCode): GlobePvpMatchRoom + removeRoom(String roomCode): void + sweepStaleRooms(): void

ScoredCandidate
- CountryBL {final}: country - ProximityLevel {final}: level - float {final}: distance

PvpGlobeBlitzEngineBL
- CountryController {final}: cc - double {final}: durationSeconds - List: queue - long: matchStartTimeMillis - boolean: matchOver - long {static final}: PAUSE_MAX_MILLIS - boolean: paused - int: pausedBySlot - long: pauseStartedAtMillis - long: pausedMillisTotal - boolean: pause1Used - boolean: pause2Used - int {static final}: NONE_ORDINAL - int: queueIndex1 - int: attemptsThisFlag1 - double: currentFlagStartElapsed1 - int: bestOrdinal1 - int: correctCount1 - List {final}: history1 - int: queueIndex2 - int: attemptsThisFlag2 - double: currentFlagStartElapsed2 - int: bestOrdinal2 - int: correctCount2 - List {final}: history2
+ {synchronized} startMatch(): void + {synchronized} submitGuess(int slot, String countryName): GuessResultGlobeBL + {synchronized} giveUpFlag(int slot): String - advanceToNextFlag(int slot): void + {synchronized} refreshState(): void + {synchronized} pauseMatch(int slot): boolean + {synchronized} resumeMatch(): void + {synchronized} refreshPauseTimeout(): void + hasUsedPause(int slot): boolean + getPauseRemainingMillis(): long + getElapsedSeconds(): double + getTimeRemainingSeconds(): double + getCorrectCount(int slot): int + getAttemptsThisFlag(int slot): int + getProgressLevel(int slot): ProximityLevel + getHistory(int slot): List + getCurrentTarget(int slot): CountryBL + getQueueFlagAt(int index): CountryBL + getMatchWinner(): PvpRoundWinner

PvpGlobeMatchEngineBL

- CountryController {final}: cc
- int {final}: pointsToWin
- int: score1
- int: score2
- int: roundNumber
- boolean: matchOver
- long: matchStartTimeMillis
- long: roundStartTimeMillis
- long {static final}: PAUSE_MAX_MILLIS
- boolean: paused
- int: pausedBySlot
- long: pauseStartedAtMillis
- long: pausedMillisThisRound
- long: pausedMillisTotalMatch
- boolean: pause1Used
- boolean: pause2Used
- long {static final}: READY_MAX_MILLIS
- boolean: ready1
- boolean: ready2
- long: roundOverAtMillis
- int {static final}: GRACE_SECONDS_PER_HINT
- int: hints1UsedThisRound
- int: hints2UsedThisRound
- Set {final}: revealedLetterPositions1
- Set {final}: revealedLetterPositions2
- CountryBL: currentTarget
- int: attempts1
- int: attempts2
- double: lastRoundDurationSeconds
- PvpRoundWinner: provisionalWinner
- double: graceDeadlineElapsedSeconds
- int {static final}: NONE_ORDINAL
- int: bestOrdinal1
- int: bestOrdinal2
- PvpRoundWinner: currentRoundWinner
- List {final}: roundHistory

- + {synchronized} startMatch(): void
- + {synchronized} advanceToNextRound(): void
- startNextRound(): void
- selectRandomCountry(): CountryBL
- + {synchronized} submitGuess(int slot, String countryName): GuessResultGlobeBL
- + {synchronized} giveUpRound(int slot): void
- finishRound(): void
- + {synchronized} markReady(int slot): boolean
- + isReady(int slot): boolean
- readyDeadlinePassed(): boolean
- + getReadyRemainingMillis(): long
- + {synchronized} useHint(int slot): String
- + getHintMaskedName(int slot): String
- + getHintsUsedThisRound(int slot): int
- + getGraceRemainingSeconds(): double
- + {synchronized} refreshGraceTimeout(): void
- + {synchronized} pauseMatch(int slot): boolean
- + {synchronized} resumeMatch(): void
- + {synchronized} refreshPauseTimeout(): void
- + hasUsedPause(int slot): boolean
- + getPauseRemainingMillis(): long
- + getRoundElapsedSeconds(): double
- + getMatchElapsedSeconds(): double
- + getScore(int slot): int
- + getAttempts(int slot): int
- + getProgressLevel(int slot): ProximityLevel
- + getBestOf(): int
- + isRoundOver(): boolean
- + getMatchWinner(): PvpRoundWinner

CapitalBL

CapitalAiOpponentPlan

- double {final}: answerTimeSeconds
- boolean {final}: correct

+ CapitalAiOpponentPlan(double answerTimeSeconds, boolean correct)
+ getAnswerTimeSeconds(): double
+ hasAnswered(double elapsedSeconds): boolean
+ isCorrect(): boolean

CapitalAiOpponentPlanFactory

- Random {static final}: RANDOM

+ {static} createPlan(AiSkillLevel level): CapitalAiOpponentPlan

CapitalQuizMode (enum)

<> CapitalQuizMode
FLAG_TO_CAPITAL
CAPITAL_TO_COUNTRY

CapitalQuestionOptions

- List {final}: optionTexts
- List {final}: optionCountries
- int {final}: correctIndex

+ CapitalQuestionOptions(List optionTexts, List optionCountries, int correctIndex)
+ getOptionTexts(): List
+ getOptionCountries(): List
+ getCorrectIndex(): int

CapitalOptionsBuilder

- int {static final}: OPTION_COUNT

+ CapitalOptionsBuilder()
+ {static} eligibleTargets(CountryController cc): List
+ {static} pickRandomTarget(CountryController cc): CountryBL
+ {static} build(CountryController cc, CapitalQuizMode mode, CountryBL target): CapitalQuestionOptions
+ {static} answerLabel(CapitalQuizMode mode, CountryBL c): String

CapitalRoundResult

- int {final}: roundNumber
- String {final}: promptText
- String {final}: correctAnswerText
- String {final}: humanPickedText
- boolean {final}: humanCorrect
- boolean {final}: aiCorrect
- String {final}: targetFlagPath
- double {final}: humanTimeSeconds
- double {final}: aiTimeSeconds

+ CapitalRoundResult(int roundNumber, String promptText, String correctAnswerText, String humanPickedText, boolean humanCorrect, boolean aiCorrect, String targetFlagPath, double humanTimeSeconds, double aiTimeSeconds)
+ getRoundNumber(): int
+ getPromptText(): String
+ getCorrectAnswerText(): String
+ getHumanPickedText(): String
+ isHumanCorrect(): boolean
+ isAiCorrect(): boolean
+ getTargetFlagPath(): String
+ getHumanTimeSeconds(): double
+ getAiTimeSeconds(): double

CapitalQuizEngineBL

- int {static final}: STAGNATION_ROUNDS_PER_POINT
- int {static final}: MISSED_TARGET_WEIGHT
- CountryController {final}: cc
- CapitalQuizMode {final}: mode
- AiSkillLevel {final}: aiLevel
- int {final}: pointsToWin
- int: humanScore
- int: aiScore
- int: roundNumber
- boolean: matchOver
- long: matchStartTimeMillis
- long: roundStartTimeMillis
- boolean: paused
- long: pauseStartedAtMillis
- long: pausedMillisThisRound
- long: pausedMillisTotalMatch
- CountryBL: currentTarget
- List: currentOptions
- List: currentOptionCountries
- int: correctOptionIndex
- int: humanPickedIndex
- boolean: currentRoundOver
- Boolean: lastHumanCorrect
- Boolean: lastAiCorrect
- double: lastHumanTimeSeconds
- CapitalAiOpponentPlan: aiPlan
- List {final}: roundHistory
- Set {final}: missedTargets

- + CapitalQuizEngineBL(CountryController cc, CapitalQuizMode mode, AiSkillLevel aiLevel, int pointsToWin)
- + {synchronized} startMatch(): void
- + {synchronized} advanceToNextRound(): void
- startNextRound(): void
- pickTarget(): CountryBL
- pickCatchUpTarget(): CountryBL
- buildOptions(CountryBL target): void
- + {synchronized} submitAnswer(int optionIndex): boolean
- resolveRound(boolean humanCorrect): void
- finishRound(): void
- + {synchronized} pauseMatch(): void
- + {synchronized} resumeMatch(): void
- + isPaused(): boolean
- + getRoundElapsedSeconds(): double
- + getMatchElapsedSeconds(): double
- + getMode(): CapitalQuizMode
- + getHumanScore(): int
- + getAiScore(): int
- + getRoundNumber(): int
- + getPointsToWin(): int
- + isMatchOver(): boolean
- + isRoundOver(): boolean
- + isLastHumanCorrect(): Boolean
- + isLastAiCorrect(): Boolean
- + getMatchWinner(): RoundWinner
- + getCurrentTarget(): CountryBL
- + getCurrentOptions(): List
- + getCurrentOptionCountries(): List
- + getCorrectOptionIndex(): int
- + getHumanPickedIndex(): int
- + getAiLevel(): AiSkillLevel
- + getRoundHistory(): List
- + isAiAnswered(): boolean

CapitalBlitzQuestionResult

- int {final}: order
- String {final}: promptText
- String {final}: correctAnswerText
- boolean {final}: correct
- double {final}: timeTakenSeconds
- String {final}: targetFlagPath

- + CapitalBlitzQuestionResult(int order, String promptText, String correctAnswerText, boolean correct, double timeTakenSeconds, String targetFlagPath)
- + getOrder(): int
- + getPromptText(): String
- + getCorrectAnswerText(): String
- + isCorrect(): boolean
- + getTimeTakenSeconds(): double
- + getTargetFlagPath(): String

CapitalBlitzEngineBL

- CountryController {final}: cc
- CapitalQuizMode {final}: mode
- AiSkillLevel {final}: aiLevel
- double {final}: durationSeconds
- List: queue
- long: matchStartTimeMillis
- boolean: matchOver
- boolean: paused
- long: pauseStartedAtMillis
- long: pausedMillisTotal
- int: humanQueueIndex
- double: humanCurrentQuestionStartElapsed
- int: humanScore
- CapitalQuestionOptions: humanCurrentOptions
- List {final}: humanHistory
- int: aiQueueIndex
- double: aiCurrentQuestionStartElapsed
- CapitalAiOpponentPlan: aiPlan
- int: aiScore
- List {final}: aiHistory

- + CapitalBlitzEngineBL(CountryController cc, CapitalQuizMode mode, AiSkillLevel aiLevel, double durationSeconds)
- + {synchronized} startMatch(): void
- + {synchronized} submitAnswer(int optionIndex): Boolean
- advanceHumanToNextQuestion(): void
- promptText(CountryBL target): String
- + {synchronized} refreshAiProgress(): void
- + {synchronized} pauseMatch(): void
- + {synchronized} resumeMatch(): void
- + isPaused(): boolean
- + getElapsedSeconds(): double
- + getTimeRemainingSeconds(): double
- + getDurationSeconds(): double
- + getMode(): CapitalQuizMode
- + isMatchOver(): boolean
- + getHumanScore(): int
- + getAiScore(): int
- + getCurrentHumanTarget(): CountryBL
- + getHumanCurrentOptions(): CapitalQuestionOptions
- + getHumanHistory(): List
- + getAiHistory(): List
- + getQueueTargetAt(int index): CountryBL
- + getAiLevel(): AiSkillLevel
- + getMatchWinner(): RoundWinner

CapitalPvpMatchRoom

- String {final}: roomCode
- CapitalQuizMode {final}: mode
- PvpGameMode {final}: gameMode
- int {final}: bestOf
- int {final}: pointsToWin
- int {final}: durationSeconds
- CountryController {final}: cc
- String {final}: player1Token
- String {volatile}: player2Token
- PvpRoomStatus {volatile}: status
- long {volatile}: lastActivityMillis
- PvpCapitalQuizEngineBL {volatile}: engine
- PvpCapitalBlitzEngineBL {volatile}: blitzEngine

- + CapitalPvpMatchRoom(String roomCode, CapitalQuizMode mode, PvpGameMode gameMode, int bestOf, int durationSeconds, String player1Token, CountryController cc)
- + {static} createBestOf(String roomCode, CapitalQuizMode mode, int bestOf, String player1Token, CountryController cc): CapitalPvpMatchRoom
- + {static} createBlitz(String roomCode, CapitalQuizMode mode, int durationSeconds, String player1Token, CountryController cc): CapitalPvpMatchRoom
- + {synchronized} joinPlayer2(String token): boolean
- + slotForToken(String token): int
- + touch(): void
- + isStale(long staleMillis): boolean
- + getStatus(): PvpRoomStatus
- + getRoomCode(): String
- + getMode(): CapitalQuizMode
- + getGameMode(): PvpGameMode
- + getBestOf(): int
- + getPointsToWin(): int
- + getDurationSeconds(): int
- + getPlayer1Token(): String
- + getPlayer2Token(): String
- + getEngine(): PvpCapitalQuizEngineBL
- + getBlitzEngine(): PvpCapitalBlitzEngineBL

CapitalQuizRoomRegistryBL

- long {static final}: STALE_ROOM_MILLIS
- String {static final}: CODE_CHARS
- int {static final}: CODE_LENGTH
- CountryController {final}: cc
- Map {final}: rooms
- Random {final}: random

- + CapitalQuizRoomRegistryBL(CountryController cc)
- + createRoom(CapitalQuizMode mode, int bestOf, String player1Token): CapitalPvpMatchRoom
- + createBlitzRoom(CapitalQuizMode mode, int durationSeconds, String player1Token): CapitalPvpMatchRoom
- generateCode(): String
- + getRoom(String roomCode): CapitalPvpMatchRoom
- + removeRoom(String roomCode): void
- + @Scheduled(fixedRate = 30_000) sweepStaleRooms(): void

PvpCapitalQuizEngineBL

- long {static final}: ANSWER_TIMEOUT_MILLIS
- long {static final}: NEXT_ROUND_DELAY_MILLIS
- CountryController {final}: cc
- CapitalQuizMode {final}: mode
- int {final}: pointsToWin
- int: score1
- int: score2
- int: roundNumber
- boolean: matchOver
- long: matchStartTimeMillis
- long: roundStartTimeMillis
- long {static final}: PAUSE_MAX_MILLIS
- boolean: paused
- int: pausedBySlot
- long: pauseStartedAtMillis
- long: pausedMillisThisRound
- long: pausedMillisTotalMatch
- boolean: pause1Used
- boolean: pause2Used
- CountryBL: currentTarget
- CapitalQuestionOptions: currentOptions
- Integer: picked1
- Integer: picked2
- boolean: roundResolved
- long: roundResolvedAtMillis
- Boolean: lastPlayer1Correct
- Boolean: lastPlayer2Correct
- List {final}: roundHistory

- + PvpCapitalQuizEngineBL(CountryController cc, CapitalQuizMode mode, int pointsToWin)
- + {synchronized} startMatch(): void
- + {synchronized} refreshRound(): void
- startNextRound(): void
- + {synchronized} submitAnswer(int slot, int optionIndex): void
- resolveRound(): void
- + {synchronized} pauseMatch(int slot): boolean
- + {synchronized} resumeMatch(): void
- + {synchronized} refreshPauseTimeout(): void
- + isPaused(): boolean
- + getPausedBySlot(): int
- + hasUsedPause(int slot): boolean
- + getPauseRemainingMillis(): long
- getRoundElapsedMillis(): long
- + getRoundElapsedSeconds(): double
- + getMatchElapsedSeconds(): double
- + getAnswerTimeoutRemainingSeconds(): double
- + getNextRoundRemainingSeconds(): double
- + getMode(): CapitalQuizMode
- + getScore(int slot): int
- + hasAnswered(int slot): boolean
- + getPicked(int slot): Integer
- + isCorrect(int slot): Boolean
- + getRoundNumber(): int
- + getPointsToWin(): int
- + isMatchOver(): boolean
- + isRoundOver(): boolean
- + getMatchWinner(): PvpRoundWinner
- + getCurrentTarget(): CountryBL
- + getCurrentOptions(): CapitalQuestionOptions
- + getRoundHistory(): List

PvpCapitalRoundResult

- int {final}: roundNumber
- String {final}: promptText
- String {final}: correctAnswerText
- String {final}: targetFlagPath
- String {final}: player1PickedText
- String {final}: player2PickedText
- boolean {final}: player1Correct
- boolean {final}: player2Correct

+ PvpCapitalRoundResult(int roundNumber, String promptText, String correctAnswerText, String targetFlagPath, String player1PickedText, String player2PickedText, boolean player1Correct, boolean player2Correct)

- + getRoundNumber(): int
- + getPromptText(): String
- + getCorrectAnswerText(): String
- + getTargetFlagPath(): String
- + getPlayer1PickedText(): String
- + getPlayer2PickedText(): String
- + isPlayer1Correct(): boolean
- + isPlayer2Correct(): boolean

PvpCapitalBlitzEngineBL

- CountryController {final}: cc
- CapitalQuizMode {final}: mode
- double {final}: durationSeconds
- List: queue
- long: matchStartTimeMillis
- boolean: matchOver
- long {static final}: PAUSE_MAX_MILLIS
- boolean: paused
- int: pausedBySlot
- long: pauseStartedAtMillis
- long: pausedMillisTotal
- boolean: pause1Used
- boolean: pause2Used
- int: queueIndex1
- double: questionStartElapsed1
- int: score1
- CapitalQuestionOptions: currentOptions1
- List {final}: history1
- int: queueIndex2
- double: questionStartElapsed2
- int: score2
- CapitalQuestionOptions: currentOptions2
- List {final}: history2

+ PvpCapitalBlitzEngineBL(CountryController cc, CapitalQuizMode mode, double durationSeconds)

- + {synchronized} startMatch(): void
- + {synchronized} submitAnswer(int slot, int optionIndex): Boolean
- advanceToNextQuestion(int slot): void
- promptText(CountryBL target): String
- + {synchronized} refreshState(): void
- + {synchronized} pauseMatch(int slot): boolean
- + {synchronized} resumeMatch(): void
- + {synchronized} refreshPauseTimeout(): void
- + isPaused(): boolean
- + getPausedBySlot(): int
- + hasUsedPause(int slot): boolean
- + getPauseRemainingMillis(): long
- + getElapsedSeconds(): double
- + getTimeRemainingSeconds(): double
- + getDurationSeconds(): double
- + getMode(): CapitalQuizMode
- + isMatchOver(): boolean
- + getScore(int slot): int
- + getHistory(int slot): List
- + getCurrentTarget(int slot): CountryBL
- + getCurrentOptions(int slot): CapitalQuestionOptions
- + getMatchWinner(): PvpRoundWinner

CapitalGlobeBL

CapitalGlobeAiOpponentPlanFactory

- Random {static final}: RANDOM

+ {static} createPlan(AiSkillLevel level): CapitalAiOpponentPlan

CapitalGlobeTargetPicker

- Set {static final}: TOO_SMALL_FOR_GLOBE
- Set {static final}: NO_CLICKABLE_POLYGON

+ CapitalGlobeTargetPicker()
+ {static} eligibleTargets(CountryController cc): List
+ {static} pickRandomTarget(CountryController cc):
CountryBL

CapitalGlobeEngineBL

- CountryController {final}: cc
- CountryBL: targetCountry
- int: attempts
- boolean: gameOver

+ CapitalGlobeEngineBL(CountryController cc)
+ startNewGame(): void
+ guess(String countryName): GuessResultGlobeBL
+ giveUp(): void
+ isGameOver(): boolean
+ getAttempts(): int
+ getTargetCountry(): CountryBL

CapitalGlobeRoundResult

- int {final}: roundNumber
- String {final}: targetCountryName
- String {final}: targetCapital
- String {final}: targetFlagPath
- boolean {final}: humanCorrect
- boolean {final}: aiCorrect
- int {final}: humanAttempts
- double {final}: humanTimeSeconds
- double {final}: aiTimeSeconds

+ CapitalGlobeRoundResult(int roundNumber, String targetCountryName, String targetCapital, String targetFlagPath, boolean humanCorrect, boolean aiCorrect, int humanAttempts, double humanTimeSeconds, double aiTimeSeconds)
+ getRoundNumber(): int
+ getTargetCountryName(): String
+ getTargetCapital(): String
+ getTargetFlagPath(): String
+ isHumanCorrect(): boolean
+ isAiCorrect(): boolean
+ getHumanAttempts(): int
+ getHumanTimeSeconds(): double
+ getAiTimeSeconds(): double

CapitalGlobeMatchEngineBL

- CountryController {final}: cc
- AiSkillLevel {final}: aiLevel
- int {final}: pointsToWin
- int: humanScore
- int: aiScore
- int: roundNumber
- boolean: matchOver
- long: matchStartTimeMillis
- long: roundStartTimeMillis
- boolean: paused
- long: pauseStartedAtMillis
- long: pausedMillisThisRound
- long: pausedMillisTotalMatch
- CountryBL: currentTarget
- int: humanAttemptsThisRound
- boolean: currentRoundOver
- Boolean: lastHumanCorrect
- Boolean: lastAiCorrect
- double: lastHumanTimeSeconds
- CapitalAiOpponentPlan: aiPlan
- List {final}: roundHistory

- + CapitalGlobeMatchEngineBL(CountryController cc, AiSkillLevel aiLevel, int pointsToWin)
- + {synchronized} startMatch(): void
- + {synchronized} advanceToNextRound(): void
- startNextRound(): void
- + {synchronized} humanGuess(String countryName): GuessResultGlobeBL
- + {synchronized} giveUpRound(): void
- resolveRound(boolean humanCorrect): void
- finishRound(): void
- + {synchronized} pauseMatch(): void
- + {synchronized} resumeMatch(): void
- + isPaused(): boolean
- + getRoundElapsedSeconds(): double
- + getMatchElapsedSeconds(): double
- + getHumanScore(): int
- + getAiScore(): int
- + getRoundNumber(): int
- + getPointsToWin(): int
- + isMatchOver(): boolean
- + isRoundOver(): boolean
- + isLastHumanCorrect(): Boolean
- + isLastAiCorrect(): Boolean
- + getMatchWinner(): RoundWinner
- + getCurrentTarget(): CountryBL
- + getHumanAttemptsThisRound(): int
- + getAiLevel(): AiSkillLevel
- + getRoundHistory(): List
- + isAiAnswered(): boolean

CapitalGlobeBlitzQuestionResult

- int {final}: order
- String {final}: targetCountryName
- String {final}: targetCapital
- String {final}: targetFlagPath
- boolean {final}: correct
- int {final}: attempts
- double {final}: timeTakenSeconds

- + CapitalGlobeBlitzQuestionResult(int order, String targetCountryName, String targetCapital, String targetFlagPath, boolean correct, int attempts, double timeTakenSeconds)
- + getOrder(): int
- + getTargetCountryName(): String
- + getTargetCapital(): String
- + getTargetFlagPath(): String
- + isCorrect(): boolean
- + getAttempts(): int
- + getTimeTakenSeconds(): double

CapitalGlobeBlitzEngineBL

- CountryController {final}: cc
- AiSkillLevel {final}: aiLevel
- double {final}: durationSeconds
- List: queue
- long: matchStartTimeMillis
- boolean: matchOver
- boolean: paused
- long: pauseStartedAtMillis
- long: pausedMillisTotal
- int: humanQueueIndex
- int: humanAttemptsThisTarget
- double: humanCurrentTargetStartElapsed
- int: humanScore
- List {final}: humanHistory
- int: aiQueueIndex
- double: aiCurrentTargetStartElapsed
- CapitalAiOpponentPlan: aiPlan
- int: aiScore
- List {final}: aiHistory

- + CapitalGlobeBlitzEngineBL(CountryController cc, AiSkillLevel aiLevel, double durationSeconds)
- + {synchronized} startMatch(): void
- + {synchronized} humanGuess(String countryName): GuessResultGlobeBL
- + {synchronized} humanSkipTarget(): String
- advanceHumanToNextTarget(): void
- + {synchronized} refreshAiProgress(): void
- + {synchronized} pauseMatch(): void
- + {synchronized} resumeMatch(): void
- + isPaused(): boolean
- + getElapsedSeconds(): double
- + getTimeRemainingSeconds(): double
- + getDurationSeconds(): double
- + isMatchOver(): boolean
- + getHumanScore(): int
- + getAiScore(): int
- + getCurrentHumanTarget(): CountryBL
- + getHumanAttemptsThisTarget(): int
- + getHumanHistory(): List
- + getAiHistory(): List
- + getAiLevel(): AiSkillLevel
- + getMatchWinner(): RoundWinner

CapitalGlobePvpMatchRoom

- String {final}: roomCode
- PvpGameMode {final}: gameMode
- int {final}: bestOf
- int {final}: pointsToWin
- int {final}: durationSeconds
- CountryController {final}: cc
- String {final}: player1Token
- String {volatile}: player2Token
- PvpRoomStatus {volatile}: status
- long {volatile}: lastActivityMillis
- PvpCapitalGlobeMatchEngineBL {volatile}: engine
- PvpCapitalGlobeBlitzEngineBL {volatile}: blitzEngine

- + CapitalGlobePvpMatchRoom(String roomCode, PvpGameMode gameMode, int bestOf, int durationSeconds, String player1Token, CountryController cc)
- + {static} createBestOf(String roomCode, int bestOf, String player1Token, CountryController cc): CapitalGlobePvpMatchRoom
- + {static} createBlitz(String roomCode, int durationSeconds, String player1Token, CountryController cc): CapitalGlobePvpMatchRoom
- + {synchronized} joinPlayer2(String token): boolean
- + slotForToken(String token): int
- + touch(): void
- + isStale(long staleMillis): boolean
- + getStatus(): PvpRoomStatus
- + getRoomCode(): String
- + getGameMode(): PvpGameMode
- + getBestOf(): int
- + getPointsToWin(): int
- + getDurationSeconds(): int
- + getPlayer1Token(): String
- + getPlayer2Token(): String
- + getEngine(): PvpCapitalGlobeMatchEngineBL
- + getBlitzEngine(): PvpCapitalGlobeBlitzEngineBL

CapitalGlobeRoomRegistryBL

- long {static final}: STALE_ROOM_MILLIS
- String {static final}: CODE_CHARS
- int {static final}: CODE_LENGTH
- CountryController {final}: cc
- Map {final}: rooms
- Random {final}: random

- + CapitalGlobeRoomRegistryBL(CountryController cc)
- + createRoom(int bestOf, String player1Token): CapitalGlobePvpMatchRoom
- + createBlitzRoom(int durationSeconds, String player1Token): CapitalGlobePvpMatchRoom
- generateCode(): String
- + getRoom(String roomCode): CapitalGlobePvpMatchRoom
- + removeRoom(String roomCode): void
- + @Scheduled(fixedRate = 30_000) sweepStaleRooms(): void

PvpCapitalGlobeBlitzEngineBL
- CountryController {final}: cc - double {final}: durationSeconds - List: queue - long: matchStartTimeMillis - boolean: matchOver - long {static final}: PAUSE_MAX_MILLIS - boolean: paused - int: pausedBySlot - long: pauseStartedAtMillis - long: pausedMillisTotal - boolean: pause1Used - boolean: pause2Used - int: queueIndex1 - int: attemptsThisTarget1 - double: currentTargetStartElapsed1 - int: score1 - List {final}: history1 - int: queueIndex2 - int: attemptsThisTarget2 - double: currentTargetStartElapsed2 - int: score2 - List {final}: history2
+ PvpCapitalGlobeBlitzEngineBL(CountryController cc, double durationSeconds) + {synchronized} startMatch(): void + {synchronized} submitGuess(int slot, String countryName): GuessResultGlobeBL + {synchronized} skipTarget(int slot): String - advanceToNextTarget(int slot): void + {synchronized} refreshState(): void + {synchronized} pauseMatch(int slot): boolean + {synchronized} resumeMatch(): void + {synchronized} refreshPauseTimeout(): void + isPaused(): boolean + getPausedBySlot(): int + hasUsedPause(int slot): boolean + getPauseRemainingMillis(): long + getElapsedSeconds(): double + getTimeRemainingSeconds(): double + getDurationSeconds(): double + isMatchOver(): boolean + getScore(int slot): int + getAttemptsThisTarget(int slot): int + getHistory(int slot): List + getCurrentTarget(int slot): CountryBL + getQueueTargetAt(int index): CountryBL + getMatchWinner(): PvpRoundWinner

PvpCapitalGlobeRoundResult
- int {final}: roundNumber - String {final}: targetCountryName - String {final}: targetCapital - String {final}: targetFlagPath - PvpRoundWinner {final}: winner - int {final}: player1Attempts - int {final}: player2Attempts - double {final}: roundDurationSeconds
+ PvpCapitalGlobeRoundResult(int roundNumber, String targetCountryName, String targetCapital, String targetFlagPath, PvpRoundWinner winner, int player1Attempts, int player2Attempts, double roundDurationSeconds) + getRoundNumber(): int + getTargetCountryName(): String + getTargetCapital(): String + getTargetFlagPath(): String + getWinner(): PvpRoundWinner + getPlayer1Attempts(): int + getPlayer2Attempts(): int + getRoundDurationSeconds(): double

PvpCapitalGlobeMatchEngineBL

- CountryController {final}: cc
- int {final}: pointsToWin
- int: score1
- int: score2
- int: roundNumber
- boolean: matchOver
- long: matchStartTimeMillis
- long: roundStartTimeMillis
- long {static final}: PAUSE_MAX_MILLIS
- boolean: paused
- int: pausedBySlot
- long: pauseStartedAtMillis
- long: pausedMillisThisRound
- long: pausedMillisTotalMatch
- boolean: pause1Used
- boolean: pause2Used
- long {static final}: READY_MAX_MILLIS
- boolean: ready1
- boolean: ready2
- long: roundOverAtMillis
- CountryBL: currentTarget
- int: attempts1
- int: attempts2
- double: lastRoundDurationSeconds
- PvpRoundWinner: currentRoundWinner
- List {final}: roundHistory

- + PvpCapitalGlobeMatchEngineBL(CountryController cc, int pointsToWin)
- + {synchronized} startMatch(): void
- + {synchronized} advanceToNextRound(): void
- startNextRound(): void
- + {synchronized} submitGuess(int slot, String countryName): GuessResultGlobeBL
- + {synchronized} giveUpRound(int slot): void
- finishRound(): void
- + {synchronized} markReady(int slot): boolean
- + isReady(int slot): boolean
- readyDeadlinePassed(): boolean
- + getReadyRemainingMillis(): long
- + getLastRoundDurationSeconds(): double
- + {synchronized} pauseMatch(int slot): boolean
- + {synchronized} resumeMatch(): void
- + {synchronized} refreshPauseTimeout(): void
- + isPaused(): boolean
- + getPausedBySlot(): int
- + hasUsedPause(int slot): boolean
- + getPauseRemainingMillis(): long
- + getRoundElapsedSeconds(): double
- + getMatchElapsedSeconds(): double
- + getScore(int slot): int
- + getAttempts(int slot): int
- + getRoundNumber(): int
- + getPointsToWin(): int
- + getBestOf(): int
- + isMatchOver(): boolean
- + isRoundOver(): boolean
- + getCurrentRoundWinner(): PvpRoundWinner
- + getMatchWinner(): PvpRoundWinner
- + getCurrentTarget(): CountryBL
- + getRoundHistory(): List

Data Access Layer

DAL

CountryRepository

- Map: codeToName
- Map: nameToCode
- List: allCountries

- + CountryRepository()
- getConnection(): Connection
- loadCountries(): void
- + getNameByCode(String code): String
- + getCodeByName(String name): String
- + getAllCountries(): List
- + getNumberOfCountries(): int

CountryController

- CountryRepository: cr

- + CountryController(CountryRepository cr)
- getConnection(): Connection
- fixImagePath(String dbPath): String
- + getCountryById(int id): CountryBL
- + getCountryByName(String countryName): CountryBL
- + getAllCountries(): List
- + getNumberOfAllCountries(): int
- CheckCountryNameExists(String countryName): boolean

CountryDAL

- int: ID
- String: countryName
- String: code
- String: flagPath
- double: latitude
- double: longitude
- String: neighborsList
- String: iso3
- String: capital

- + CountryDAL(int ID, String countryName, String code, String flagPath, double lat, double longitude, String neighbors, String iso3, String capital)
- + getID(): int
- + getCountryName(): String
- + getCode(): String
- + getFlagPath(): String
- + getLatitude(): double
- + getLongitude(): double
- + getNeighborsList(): String
- + getIso3(): String
- + getCapital(): String

Countries (table)
PK ID: INTEGER Code: TEXT ISO3: TEXT CountryName: TEXT FlagPath: TEXT Latitude: REAL Longitude: REAL neighborList: TEXT Capital: TEXT